

USCS – Universidade Municipal de São Caetano do Sul Curso: Ciência da Computação

Material de Estudos - ENADE

TEORIA DA COMPUTAÇÃO

Nome dos Integrantes do Grupo:

Eder de Souza Vanzei	8154562
Gabriel Campari Correia	8145921
Gabriel Henrique Imolene Vieira	8154947
Glenda Borges Ferreira de Carvalho	8149720
João Victor Maximiano Vieira	8151502
Lívia Mezashi	8144678

São Caetano do Sul

2026

UNIDADE 1 - Fundamentos de Conjuntos e Relações

1. Introdução

A Teoria dos Conjuntos fornece uma linguagem matemática para organizar, comparar e relacionar objetos. Em Computação, esse tipo de raciocínio aparece em diferentes situações: na definição de conjuntos de estados de um autômato, na representação de alfabetos e linguagens formais, no tratamento de permissões, em consultas a bancos de dados, em lógica e em estruturas matemáticas utilizadas por algoritmos.

Nesta unidade, o objetivo é compreender os conceitos de conjunto, elemento, subconjunto, conjunto vazio, operações entre conjuntos, conjunto universo e complemento. Em seguida, o conteúdo avança para relações binárias e para as propriedades reflexiva, simétrica, antissimétrica e transitiva. Esses tópicos formam uma base importante para as unidades seguintes de Teoria da Computação.

O conteúdo-base desta unidade corresponde às páginas 4 a 13 do slide de revisão fornecido pela turma. O presente material amplia essa base com exemplos e explicações adicionais.

2. Conjuntos: definição e notação

Um conjunto é uma coleção de objetos bem determinados, chamados elementos. Os conjuntos costumam ser representados por letras maiúsculas, como A, B, C e U, enquanto seus elementos são escritos entre chaves.

$A = \{1, 2, 3\}$

Nesse exemplo, o conjunto A possui três elementos: 1, 2 e 3. A ordem em que os elementos são escritos não altera o conjunto. Assim, {1, 2, 3} e {3, 2, 1} representam o mesmo conjunto. Também não se considera repetição de elementos: {1, 1, 2, 3, 3} representa o mesmo conjunto que {1, 2, 3}.

Para indicar que um objeto pertence a um conjunto, utiliza-se o símbolo $\in$. Para indicar que não pertence, utiliza-se $\notin$.

$$2 \in A \qquad e \qquad 5 \notin A$$

A primeira expressão informa que 2 é um elemento de A. A segunda informa que 5 não é elemento de A.

2.1 Elemento não é a mesma coisa que conjunto

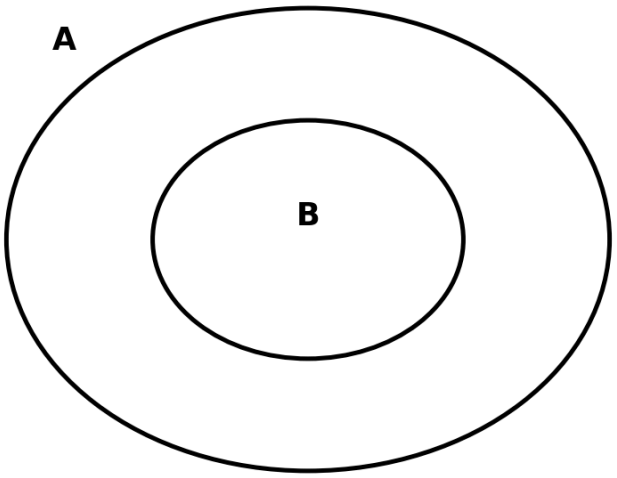
É importante distinguir um objeto de um conjunto que contém esse objeto. Por exemplo, 1 é um número, enquanto {1} é um conjunto cujo único elemento é o número 1. Essa diferença será importante quando forem utilizados os símbolos de pertencimento e de inclusão.

Expressão	Leitura	Exemplo
$x \in A$	x pertence ao conjunto A	$1 \in \{1, 2, 3\}$
$x \notin A$	x não pertence ao conjunto A	$5 \notin \{1, 2, 3\}$
$B \subseteq A$	B é subconjunto de A	$\{1, 2\} \subseteq \{1, 2, 3\}$

3. Subconjuntos

Dizemos que B é subconjunto de A quando todo elemento pertencente a B também pertence a A. A notação utilizada é $B \subseteq A$.

Se $A = \{1, 2, 3\}$ e $B = \{1, 2\}$, então $B \subseteq A$.



$B \subseteq A$: todo elemento de B também pertence a A

O diagrama representa B inteiramente dentro de A. Isso significa que não existe elemento de B fora de A.

Todo conjunto é subconjunto dele mesmo. Portanto, $A \subseteq A$ é sempre verdadeiro. Além disso, o conjunto vazio é subconjunto de qualquer conjunto.

$\varnothing \subseteq A$

3.1 Conjunto das partes

Uma ampliação útil do conceito de subconjunto é o conjunto das partes, também chamado conjunto potência. Ele é formado por todos os subconjuntos possíveis de um conjunto.

Se A = {1, 2}, então P(A) = {⌀, {1}, {2}, {1, 2}}.

Um conjunto com n elementos possui 2^n subconjuntos. Assim, um conjunto com 3 elementos possui $2^3 = 8$ subconjuntos. Embora esse conceito não esteja detalhado no slide-base, ele decorre diretamente da noção de subconjunto e ajuda a compreender a quantidade de combinações possíveis.

4. Conjunto vazio

O conjunto vazio é o conjunto que não possui elementos. Ele pode ser representado por ⌀ ou por {}.

⌀ = {}

É essencial não confundir ⌀ com {⌀}. O primeiro possui zero elementos; o segundo possui um elemento, e esse elemento é o próprio conjunto vazio.

Representação	Quantidade de elementos	Interpretação
⌀	0	conjunto sem elementos
{⌀}	1	conjunto que contém o conjunto vazio

A propriedade ⌀ ⊆ A vale para qualquer conjunto A. A razão é lógica: não existe elemento do conjunto vazio que possa violar a condição de estar em A.

5. Operações entre conjuntos

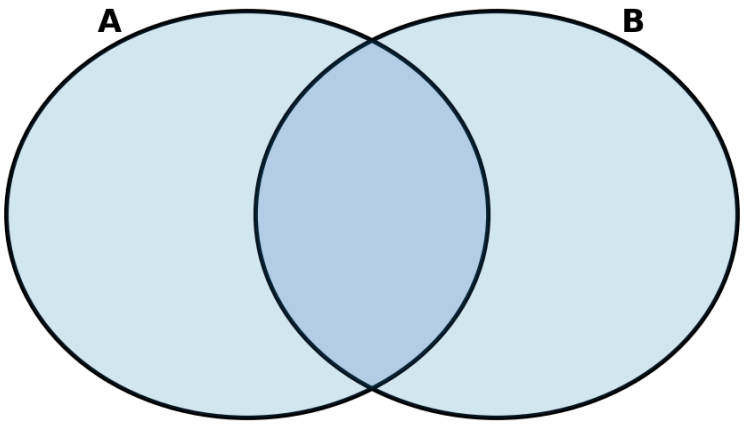
As principais operações apresentadas nesta unidade são união, interseção e diferença. Cada uma responde a uma pergunta diferente sobre a distribuição dos elementos.

5.1 União

A união de A e B, representada por A ∪ B, reúne todos os elementos que pertencem a A, a B ou a ambos. Elementos repetidos são escritos uma única vez.

A = {1, 2, 3} B = {3, 4, 5}

A ∪ B = {1, 2, 3, 4, 5}

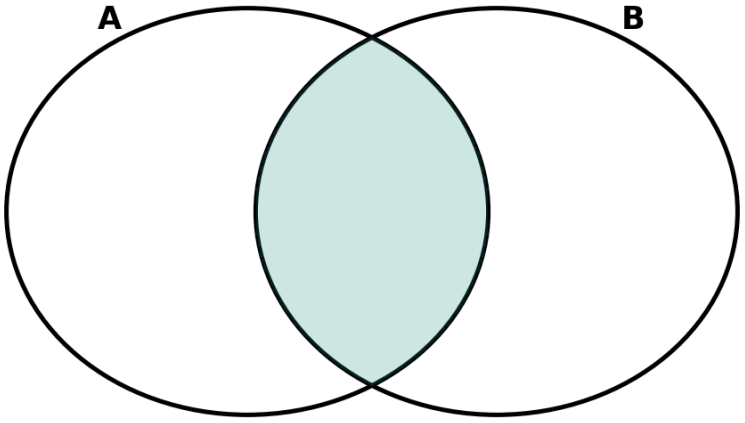


A ∪ B: toda a região pertencente a A ou a B

5.2 Interseção

A interseção, representada por A ∩ B, contém somente os elementos que pertencem simultaneamente aos dois conjuntos.

A ∩ B = {3}



A ∩ B: apenas a região comum aos dois conjuntos

Quando dois conjuntos não possuem elementos em comum, sua interseção é vazia. Nesse caso, dizemos que são conjuntos disjuntos.

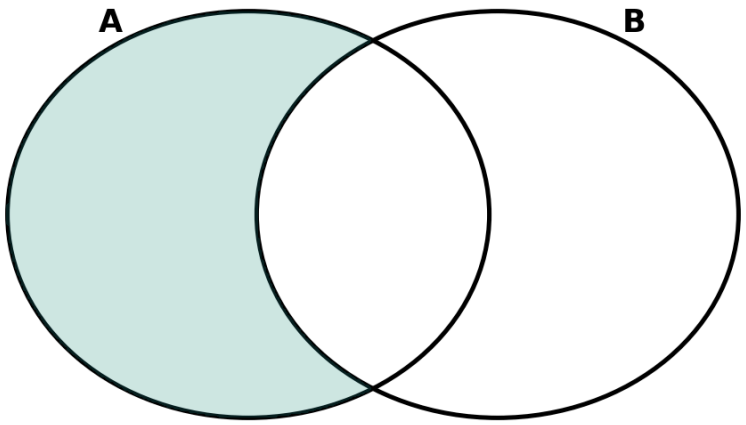
A ∩ B = ∅

5.3 Diferença

A diferença A − B contém os elementos que pertencem a A, mas não pertencem a B. A ordem importa.

A = {1, 2, 3} B = {3, 4, 5}

A − B = {1, 2} e B − A = {4, 5}



A − B: elementos que estão em A e não estão em B

Portanto, em geral, A − B ≠ B − A. Essa é uma diferença importante em relação à união e à interseção, que são comutativas.

Operação	**Ideia principal**	**Resultado no exemplo A={1,2,3}, B={3,4,5}**
A ∪ B	Tudo que está em A ou em B	{1,2,3,4,5}
A ∩ B	Apenas o que está nos dois	{3}
A − B	Está em A, mas não em B	{1,2}
B − A	Está em B, mas não em A	{4,5}

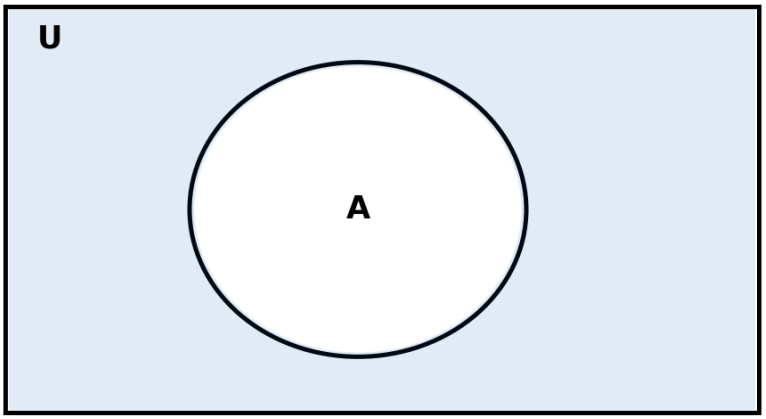
6. Conjunto universo e complemento

O conjunto universo, indicado por U, reúne todos os elementos considerados em uma determinada situação. O universo depende do problema. Por exemplo, se a análise estiver restrita aos números de 1 a 5, podemos definir U = {1, 2, 3, 4, 5}.

O complemento de A é formado pelos elementos que pertencem ao universo, mas não pertencem a A. No slide-base, o complemento é representado por A'.

$A' = U - A$

Se $U = \{1, 2, 3, 4\}$ e $A = \{1, 4\}$, então $A' = \{2, 3\}$.



$A' = U - A$: região do universo que não pertence a A

O complemento só pode ser determinado quando o universo estiver definido. Se sabemos apenas que $A = \{1, 2\}$, mas não sabemos quais elementos pertencem a U, não é possível listar A'.

7. Relações

Uma relação é uma forma de associar elementos. No contexto desta unidade, trabalhamos principalmente com relações binárias, isto é, relações formadas por pares ordenados.

$R = \{(1, 2), (1, 3)\}$

O par ordenado (1, 2) indica que 1 está relacionado a 2 segundo a regra definida por R. Em pares ordenados, a ordem importa: (1, 2) e (2, 1) não são, em geral, o mesmo par.

Formalmente, uma relação de A em B pode ser entendida como um subconjunto do produto cartesiano $A \times B$. Quando a relação é definida sobre um único conjunto A, temos $R \subseteq A \times A$. Essa formulação aparece de forma recorrente em Matemática Discreta e será útil para entender as propriedades a seguir.

$A \times B = \{(a, b) \mid a \in A \text{ e } b \in B\}$

7.1 Produto cartesiano: exemplo

$A = \{1, 2\}$ $B = \{a, b\}$

$A \times B = \{(1,a), (1,b), (2,a), (2,b)\}$

Uma relação de A em B pode selecionar alguns desses pares. Por exemplo, $R = \{(1,a), (2,b)\}$ é uma relação de A em B.

8. Propriedades das relações

As relações podem apresentar propriedades estruturais. Nesta unidade, são estudadas quatro: reflexiva, simétrica, antissimétrica e transitiva. Uma mesma relação pode possuir mais de uma dessas propriedades ao mesmo tempo.

8.1 Relação reflexiva

Uma relação R sobre A é reflexiva quando todo elemento de A se relaciona consigo mesmo.

$\forall x \in A, (x, x) \in R$

Se $A = \{1, 2, 3\}$, uma relação reflexiva precisa conter obrigatoriamente (1,1), (2,2) e (3,3). Ela pode conter outros pares também.

$R = \{(1,1), (2,2), (3,3), (1,2)\}$

Essa relação é reflexiva, pois todos os pares (x,x) exigidos estão presentes. Se faltasse, por exemplo, (3,3), ela deixaria de ser reflexiva.

> ***Estratégia de verificação:*** *liste todos os elementos de A e confira se cada um aparece no par com ele mesmo.*

8.2 Relação simétrica

Uma relação é simétrica quando todo par possui seu par invertido.

$(x, y) \in R \Rightarrow (y, x) \in R$

$R = \{(1,2), (2,1), (2,3), (3,2)\}$

Nesse exemplo, cada relação entre elementos diferentes aparece nos dois sentidos. Pares como (1,1) não causam problema, pois seu inverso é o próprio par.

Uma analogia simples é a relação 'ter a mesma idade que': se A tem a mesma idade que B, então B tem a mesma idade que A.

8.3 Relação antissimétrica

A palavra antissimétrica costuma gerar confusão. Antissimétrica não significa simplesmente 'não simétrica'. A condição é mais específica: se dois elementos se relacionam nos dois sentidos, então eles devem ser o mesmo elemento.

$(x, y) \in R \land (y, x) \in R \Rightarrow x = y$

Assim, se R contém (1,2) e (2,1), ela não é antissimétrica, pois $1 \neq 2$. Por outro lado, os pares (1,1), (2,2) e (3,3) são perfeitamente compatíveis com a antissimetria.

$R = \{(1,1), (2,2), (3,3), (1,2)\}$

A relação acima é antissimétrica: há (1,2), mas não há (2,1). Um exemplo conhecido é a relação ≤ nos números: se $x \leq y$ e $y \leq x$, necessariamente $x = y$.

> ***Importante:*** *uma relação pode ser simultaneamente simétrica e antissimétrica. Isso acontece, por exemplo, em uma relação formada apenas por pares (x,x).*

8.4 Relação transitiva

Uma relação é transitiva quando uma cadeia de dois relacionamentos exige um terceiro relacionamento.

$(x, y) \in R \land (y, z) \in R \Rightarrow (x, z) \in R$

$R = \{(1,2), (2,3), (1,3)\}$

Como 1 se relaciona com 2 e 2 se relaciona com 3, a transitividade exige que 1 se relacione com 3. O par (1,3) está presente, então essa cadeia satisfaz a propriedade.

Se R fosse apenas {(1,2), (2,3)}, ela não seria transitiva, porque faltaria o par (1,3).

9. Comparação entre as propriedades

Propriedade	Pergunta prática para verificar	Condição formal
Reflexiva	Todo elemento se relaciona consigo mesmo?	$\forall x \in A, (x,x) \in R$
Simétrica	Se existe (x,y), também existe (y,x)?	$(x,y) \in R \Rightarrow (y,x) \in R$
Antissimétrica	Há pares invertidos entre elementos diferentes?	$(x,y) \in R \land (y,x) \in R \Rightarrow x=y$
Transitiva	Se há (x,y) e (y,z), também há (x,z)?	$(x,y) \in R \land (y,z) \in R \Rightarrow (x,z) \in R$

10. Exemplo completo de análise

A = {1, 2, 3}

R = {(1,1), (2,2), (3,3), (1,2), (2,1)}

Reflexiva: sim. Os pares (1,1), (2,2) e (3,3) estão presentes.

Simétrica: sim. O par (1,2) possui o inverso (2,1). Os pares (x,x) são seus próprios inversos.

Antissimétrica: não. Existem (1,2) e (2,1), embora 1 e 2 sejam diferentes.

Transitiva: a análise deve considerar todas as cadeias possíveis. Como há (1,2) e (2,1), a transitividade exige (1,1), que existe. Como há (2,1) e (1,2), exige (2,2), que também existe. As demais composições envolvendo pares reflexivos não criam exigências ausentes. Portanto, essa relação é transitiva.

Conclusão: R é reflexiva, simétrica e transitiva, mas não é antissimétrica.

11. Erros frequentes

- Confundir $\in$ com $\subseteq$: o primeiro relaciona elemento e conjunto; o segundo relaciona conjunto e conjunto.
- Confundir $\varnothing$ com $\{\varnothing\}$: o primeiro tem zero elementos; o segundo tem um elemento.
- Assumir que $A - B = B - A$: a diferença depende da ordem.
- Achar que antissimétrica significa 'o contrário de simétrica': a definição de antissimetria é condicional e permite pares (x,x).
- Verificar transitividade observando apenas uma cadeia: é necessário conferir todas as cadeias (x,y) e (y,z) existentes.

12. Aplicação em Computação

Conjuntos e relações não são apenas conteúdos abstratos. Na Computação, conjuntos podem representar estados válidos, usuários, permissões, símbolos, palavras ou resultados de consultas. Operações de união e interseção aparecem, por exemplo, quando queremos combinar ou comparar grupos de dados.

Relações também aparecem em estruturas como grafos, dependências, ordens, equivalências, bancos de dados e modelos formais. Em Teoria da Computação, esse raciocínio prepara o estudante para compreender alfabetos, linguagens e autômatos nas unidades seguintes.

13. Questões de fixação

Questão 1. Considere $A = \{1,2,3\}$ e $B = \{2,3,4\}$. Determine $A \cup B$, $A \cap B$, $A - B$ e $B - A$.

Questão 2. Explique por que $\varnothing$ e $\{\varnothing\}$ não representam o mesmo conjunto.

Questão 3. Se $A = \{a,b,c\}$, liste todos os subconjuntos de A e confirme que a quantidade é 2^3.

Questão 4. Considere $A = \{1,2,3\}$ e $R = \{(1,1),(2,2),(3,3),(1,2),(2,1)\}$. Classifique R quanto às propriedades reflexiva, simétrica, antissimétrica e transitiva.

Questão 5. Dê um exemplo de uma relação que seja antissimétrica, mas não simétrica.

14. Questão estilo ENADE

Considere os conjuntos $A = \{1,2,3\}$, $B = \{2,3,4\}$ e a relação $R = \{(1,1),(2,2),(3,3),(1,2),(2,1)\}$ definida sobre A. Analise as afirmações:

- I. $A \cup B = \{1,2,3,4\}$.
- II. $A \cap B = \{2,3\}$.
- III. $A - B = \{1\}$.
- IV. R é reflexiva.
- V. R é simétrica.
- VI. R é antissimétrica.

A sequência correta é: V, V, V, V, V, F.

Justificativa: As três primeiras afirmações seguem diretamente das operações entre conjuntos. R é reflexiva porque contém (1,1), (2,2) e (3,3). É simétrica porque (1,2) e (2,1) aparecem simultaneamente. Não é antissimétrica porque $1 \neq 2$ e, mesmo assim, os dois sentidos da relação estão presentes.

15. Síntese da unidade

Conjuntos são coleções de elementos. A relação de pertencimento é indicada por $\in$, enquanto a inclusão entre conjuntos é indicada por $\subseteq$. O conjunto vazio, representado por $\varnothing$, não possui elementos e é subconjunto de qualquer conjunto.

As operações básicas são união, interseção e diferença. O conjunto universo delimita os elementos considerados e permite definir o complemento. Relações binárias associam elementos por meio de pares ordenados e podem possuir propriedades reflexiva, simétrica, antissimétrica e transitiva.

O domínio dessas definições é importante porque os mesmos conceitos reaparecem na formalização de linguagens, autômatos, grafos, ordens, equivalências e modelos computacionais.

UNIDADE 2 - Linguagens Formais e Expressões Regulares

1. Introdução

As linguagens formais constituem uma das bases da Teoria da Computação. Elas permitem descrever, de maneira matemática, conjuntos de cadeias de símbolos e estabelecer regras para reconhecer ou gerar essas cadeias. Esse estudo é fundamental para compreender expressões regulares, autômatos, compiladores, análise léxica, linguagens de programação e vários modelos computacionais.

Nesta unidade, o conteúdo começa pelas noções de alfabeto, palavra, comprimento e linguagem, apresenta as gramáticas formais, passa por operações sobre palavras e linguagens, apresenta a Hierarquia de Chomsky e termina com expressões regulares e seus operadores.

2. Alfabeto

Um alfabeto é um conjunto finito e não vazio de símbolos. O símbolo grego Σ (sigma) é normalmente utilizado para representar um alfabeto.

$\Sigma = \{0, 1\}$

Nesse caso, o alfabeto possui dois símbolos: 0 e 1. Outro exemplo é o conjunto dos algarismos decimais:

$\Sigma = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9\}$

Os elementos de um alfabeto são símbolos. Eles não precisam ser números: um alfabeto também pode ser formado por letras, sinais ou outros símbolos, desde que o conjunto seja finito e não vazio.

$\Sigma = \{a, b, c\}$

3. Palavra ou cadeia

Uma palavra, também chamada cadeia, é uma sequência finita de símbolos pertencentes a um alfabeto. Costuma-se utilizar w para representar uma palavra.

Σ = {0,1} w = 01111

A palavra 01111 é válida sobre esse alfabeto porque todos os símbolos que aparecem nela, 0 e 1, pertencem a Σ.

Uma palavra não é um conjunto. Em uma palavra, a ordem e a repetição importam. Assim, 01, 10 e 001 são palavras diferentes, embora utilizem os mesmos símbolos.

3.1 Comprimento de uma palavra

O comprimento de uma palavra é a quantidade de símbolos que ela possui. A notação |w| representa o comprimento da palavra w.

Se w = 100, então |w| = 3.

Se w = ababa, então |w| = 5.

3.2 Palavra vazia

A palavra vazia é a palavra que possui comprimento zero. Ela é representada por **ε** (épsilon).

|ε| = 0

A palavra vazia não possui símbolos, mas ainda é uma palavra válida. Ela exerce, na concatenação, um papel semelhante ao número 0 na soma: concatenar **ε** com uma palavra não altera a palavra.

wε = εw = w

> ***Importante:*** *não confundir **ε** com ∅. **ε** é uma palavra; ∅ representa uma linguagem sem palavras.*

4. Linguagem formal

Uma linguagem é um conjunto de palavras formadas sobre um determinado alfabeto. Ela não precisa conter todas as palavras possíveis que podem ser construídas com esse alfabeto.

Σ = {0,1}

L = {0, 00, 000, 1}

Todas as palavras de L utilizam apenas símbolos de Σ, mas várias outras palavras possíveis sobre Σ, como 01, 10 ou 111, não pertencem a L.

Formalmente, uma linguagem L sobre Σ é um subconjunto de Σ*. Essa notação será explicada adiante.

L ⊆ Σ*

5. Gramáticas formais

5.1 Ideia geral

Até aqui, uma linguagem foi descrita por enumeração (L = {0, 00, 000, 1}) ou por uma propriedade de suas palavras. Uma terceira forma é a gramática formal: um conjunto finito de regras que descreve como as palavras de uma linguagem podem ser construídas. Em vez de listar as palavras, a gramática fornece um mecanismo de geração: aplicando as regras a partir de um ponto de partida, obtêm-se exatamente as palavras da linguagem.
A ideia central pode ser resumida no fluxo:
Gramática → gera palavras → forma uma linguagem
A linguagem gerada por uma gramática G é representada por L(G).

5.2 Definição formal

Uma gramática é uma quádrupla:

G = (V, Σ, P, S)

Componente	Nome	Função
V	Variáveis (não terminais)	Símbolos auxiliares, usados apenas durante a geração. Não aparecem na palavra final. Exemplo: S, A, B.
Σ	Símbolos terminais	Símbolos finais que compõem as palavras da linguagem. Exemplo: {a, b}.
P	Produções	Regras que determinam como os símbolos são substituídos. Exemplo: S → aSb | ε.
S	Símbolo inicial	Variável especial (S ∈ V) a partir da qual se inicia a derivação.

Os conjuntos V e Σ são disjuntos: um símbolo é ou auxiliar ou terminal, nunca ambos. Por convenção, variáveis são escritas em letras maiúsculas (S, A, B) e terminais em letras minúsculas ou dígitos (a, b, 0, 1).

5.3 Produções

Uma produção tem a forma α → β, lida como "α pode ser substituído por β". Quando várias produções têm o mesmo lado esquerdo, pode-se agrupá-las com o símbolo |, que aqui significa "ou". Por exemplo:
S → aSb | ε

equivale às duas produções S → aSb e S → ε.
Esse uso do símbolo | será retomado nas expressões regulares, em que ele também indica alternativa.

5.4 Derivação

Uma derivação é uma sequência de substituições que parte de S e aplica produções até restarem somente terminais. Cada passo de substituição é indicado por ⇒.
Exemplo. Sejam G = ({S}, {a, b}, P, S) e P: S → aSb | ε. Para gerar a palavra aabb:
S ⇒ aSb ⇒ aaSbb ⇒ aabb

No primeiro e no segundo passos aplicou-se S → aSb; no último, S → ε, que eliminou a variável. Como o resultado contém apenas terminais, a derivação termina.
Repetindo o raciocínio, a gramática gera ε, ab, aabb, aaabbb, ...
Logo:

$L(G) = \{ a^n b^n \mid n \geq 0 \}$

Essa linguagem é um exemplo clássico: exige que o número de a's seja igual ao de b's. Mais adiante, será visto que ela não pode ser descrita por expressão regular nem reconhecida por autômato finito, mas é gerada por uma gramática simples.

5.5 Linguagem gerada por uma gramática

A linguagem gerada por G é o conjunto de todas as palavras formadas somente por terminais que podem ser derivadas a partir de S:

$L(G) = \{ w \in \Sigma^* \mid S \Rightarrow^* w \}$

em que $\Rightarrow^*$ indica zero ou mais passos de derivação. Note que $L(G) \subseteq \Sigma^*$, em coerência com a definição de linguagem formal apresentada na seção 4. Uma palavra só pertence a L(G) se existir ao menos uma derivação que a produza; formas intermediárias, como aSb, contêm variáveis e por isso não são palavras da linguagem.

6.Operações com palavras

6.1 Concatenação

Concatenar duas palavras significa colocá-las lado a lado, preservando a ordem.

u = abc v = de

uv = abcde

A concatenação geralmente não é comutativa. No exemplo anterior, uv = abcde, enquanto vu = deabc. Portanto, em geral, uv ≠ vu.

O comprimento da concatenação é a soma dos comprimentos:

|uv| = |u| + |v|

6.2 Potência de uma palavra

A potência de uma palavra corresponde à sua concatenação repetida com ela mesma.

w = abc

w^2 = ww = abcabc

w^3 = www = abcabcabc

Por definição, a potência zero de qualquer palavra é a palavra vazia:

w^0 = ε

Essa definição é importante porque mantém as propriedades algébricas da concatenação.

7. Σ* e o conjunto de todas as palavras

A notação Σ* representa o conjunto de todas as palavras finitas que podem ser formadas com símbolos de Σ, incluindo a palavra vazia.

Exemplo: Σ = {0,1}

Comprimento 0	ε
Comprimento 1	0, 1
Comprimento 2	00, 01, 10, 11
Comprimento 3	000, 001, 010, 011, 100, 101, 110, 111
…	…

Σ* reúne todas as palavras finitas que podem ser formadas sobre Σ, incluindo ε.

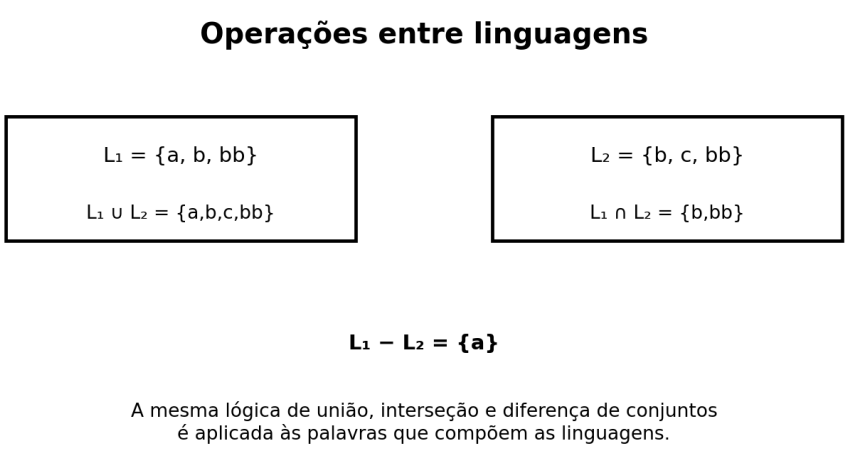
Se Σ = {0,1}, então:

Σ* = {ε, 0, 1, 00, 01, 10, 11, 000, 001, ...}

O conjunto Σ* é infinito sempre que Σ possui pelo menos um símbolo, porque não existe limite para o comprimento das palavras que podem ser formadas.

8. Operações sobre linguagens

Como uma linguagem é um conjunto de palavras, muitas operações de conjuntos podem ser aplicadas diretamente às linguagens.



8.1 União

A união reúne todas as palavras que pertencem a pelo menos uma das linguagens.

L_1 = {a,b} L_2 = {c,d}

$L_1 \cup L_2$ = {a,b,c,d}

8.2 Interseção

A interseção contém apenas as palavras que pertencem simultaneamente às duas linguagens.

L_1 = {aa,bb} L_2 = {a,bb,c}

$L_1 \cap L_2$ = {bb}

8.3 Diferença

A diferença $L_1 - L_2$ contém as palavras que pertencem a L_1, mas não pertencem a L_2.

$L_1 = \{aa,bb\}$ $L_2 = \{a,bb,c\}$

$L_1 - L_2 = \{aa\}$

> *No conteúdo-base havia um pequeno erro de símbolo no exemplo, indicando interseção no lugar de diferença. Aqui foi adotada a operação coerente com a definição: $L_1 - L_2 = \{aa\}$.*

9. Concatenação de linguagens

Também é possível concatenar linguagens. Se L_1 e L_2 são linguagens, sua concatenação é formada por todas as palavras obtidas concatenando uma palavra de L_1 com uma palavra de L_2.

$L_1 = \{a,b\}$ $L_2 = \{0,1\}$

$L_1L_2 = \{a0, a1, b0, b1\}$

Esse conceito será importante para entender a potência de linguagens e os fechos.

10. Potência de uma linguagem

A potência L^n representa a concatenação da linguagem L com ela mesma n vezes.

$L^0 = \{\varepsilon\}$

$L^1 = L$

$L^2 = LL$

Por exemplo, se $L = \{a,b\}$:

$L^2 = \{aa, ab, ba, bb\}$

11. Fecho de Kleene

O fecho de Kleene de uma linguagem L, representado por L*, reúne todas as palavras que podem ser obtidas concatenando zero ou mais palavras de L.

$L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \ldots$

Como $L^0 = \{\varepsilon\}$, a palavra vazia sempre pertence a L*.

$\varepsilon \in L^*$

Se L = {a,b}, então L* não é apenas {ε,a,b}; ele também contém todas as concatenações possíveis:

L* = {ε, a, b, aa, ab, ba, bb, aaa, aab, ...}

> *Essa é uma ampliação importante em relação à forma resumida apresentada nos slides: o fecho de Kleene contém palavras de qualquer comprimento finito construídas por concatenações de elementos de L.*

12. Fecho positivo

O fecho positivo, representado por L+, reúne as palavras obtidas por uma ou mais concatenações de palavras de L.

$L+ = L^1 \cup L^2 \cup L^3 \cup \ldots$

Quando ε não pertence originalmente a L, uma maneira útil de relacionar os dois fechos é:

$L^* = L+ \cup \{\varepsilon\}$

Se L = {a,b}, então:

L+ = {a, b, aa, ab, ba, bb, aaa, ...}

Assim, a diferença conceitual central é que L* permite zero repetições e, por isso, inclui ε; L+ exige pelo menos uma repetição.

13. Complemento de uma linguagem

O complemento de uma linguagem depende do universo de palavras considerado. Em Linguagens Formais, esse universo costuma ser Σ*.

Σ = {a,b}

L = {a,ab}

$L^c = \Sigma^* - L$

Portanto, o complemento de L contém todas as palavras sobre Σ que não pertencem a L, como ε, b, aa, bb, ba, aaa e muitas outras.

Sem especificar Σ, o complemento fica ambíguo, pois não sabemos quais palavras fazem parte do universo.

14. Hierarquia de Chomsky

A Hierarquia de Chomsky organiza classes de linguagens formais de acordo com o poder das regras necessárias para descrevê-las e dos modelos computacionais capazes de reconhecê-las.



A classificação tradicional possui quatro níveis:

Tipo	Classe de linguagem	Modelo associado (visão geral)
Tipo 3	Linguagens regulares	Autômatos finitos / expressões regulares
Tipo 2	Linguagens livres de contexto	Autômatos de pilha
Tipo 1	Linguagens sensíveis ao contexto	Autômatos limitados linearmente
Tipo 0	Linguagens recursivamente enumeráveis	Máquinas de Turing

As linguagens regulares formam a classe mais restrita entre essas quatro. Conforme avançamos para níveis externos da hierarquia, aumenta o poder de descrição e reconhecimento.

Essa visão será retomada nas próximas unidades, quando forem estudados autômatos finitos, autômatos de pilha e máquinas de Turing.

15. Expressões regulares

Uma expressão regular é uma representação formal usada para descrever uma linguagem regular sem precisar enumerar individualmente todas as suas palavras.

Por exemplo, em vez de escrever um conjunto infinito como {a, aa, aaa, aaaa, ...}, podemos utilizar a expressão:

a^+

Essa expressão descreve uma ou mais ocorrências consecutivas de a.

Expressões regulares são especialmente importantes porque fornecem uma forma compacta de descrever padrões. Em Teoria da Computação, elas estão relacionadas diretamente às linguagens regulares e aos autômatos finitos.

16. Operadores básicos de expressões regulares

Símbolo	Significado	Exemplo	Interpretação
|	União / alternativa	a|b	aceita a ou b
*	Fecho de Kleene	a*	zero ou mais ocorrências de a
+	Fecho positivo	a+	uma ou mais ocorrências de a
( )	Agrupamento	(ab)*	repete o bloco ab zero ou mais vezes
Concatenação	Justaposição	ab	a seguido de b
ε	Palavra vazia	a|ε	aceita a ou a palavra vazia
∅	Linguagem vazia	a∅	não gera palavra alguma
Σ	Alfabeto	Σ={0,1}	símbolos permitidos
Σ*	Todas as palavras sobre Σ	{0,1}*	qualquer palavra binária finita, inclusive ε

16.1 Diferença entre ε e ∅

Essa distinção é fundamental:

- ε é uma palavra de comprimento zero.
- ∅ é uma linguagem que não contém palavra alguma.

{ε} ≠ ∅

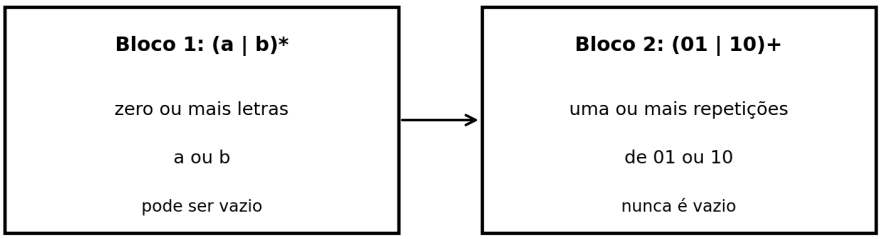
A linguagem {ε} possui exatamente uma palavra: a palavra vazia. Já ∅ possui zero palavras.

17. Interpretando uma expressão regular

Σ = {a,b,0,1}

(a | b)* (01 | 10)+

(a | b)* (01 | 10)+



Exemplos aceitos: 01, 10, a01, bb10, ab0110, ba1010, ...

A expressão pode ser dividida em dois blocos.

Primeiro bloco: (a|b)*. O operador | indica alternativa, então cada símbolo pode ser a ou b. O * permite zero ou mais repetições. Por isso, esse bloco pode gerar ε, a, b, aa, ab, ba, bb, aba, aabb e muitas outras combinações.

Segundo bloco: (01|10)+. O operador + exige uma ou mais repetições do bloco. Cada repetição deve ser 01 ou 10. Portanto, esse trecho pode gerar 01, 10, 0101, 0110, 1001, 1010 e outras concatenações desses dois blocos.

Como os dois blocos estão lado a lado, ocorre concatenação. A palavra completa pode começar com qualquer sequência de a e b, inclusive nenhuma letra, mas deve terminar com pelo menos um bloco 01 ou 10.

Exemplos aceitos:

- 01
- 10
- a01
- bb10
- ab0110
- ba1010

Exemplos não aceitos:

- aab — não termina com 01 ou 10.
- ε — o segundo bloco exige pelo menos uma ocorrência.
- 001 — não pode ser decomposta corretamente em uma parte de a/b seguida de blocos 01 ou 10.
- a11 — 11 não corresponde a 01 ou 10.

18. Como resolver exercícios de expressões regulares

Uma estratégia eficiente é interpretar a expressão da esquerda para a direita, respeitando agrupamentos e operadores.

- Identifique o alfabeto.
- Separe os blocos entre parênteses.
- Localize operadores *, + e |.
- Determine quais partes podem gerar ε.
- Observe as concatenações obrigatórias.
- Teste palavras candidatas decompondo-as conforme os blocos da expressão.

Por exemplo, para verificar se ab0110 pertence à linguagem descrita por (a|b)*(01|10)+, podemos separar ab + 01 + 10. A parte ab é gerada pelo primeiro bloco e 01 e 10 são duas repetições válidas do segundo. Logo, a palavra é aceita.

19. Erros frequentes

- Confundir alfabeto com linguagem: o alfabeto contém símbolos; a linguagem contém palavras.
- Confundir ε com ∅: ε é uma palavra; ∅ é uma linguagem vazia.
- Achar que L* contém apenas os elementos originais de L mais ε. Na verdade, contém todas as concatenações finitas possíveis.
- Achar que L+ significa simplesmente "L sem ε". A definição correta é uma ou mais concatenações de palavras de L.
- Esquecer que a concatenação de palavras depende da ordem.
- Interpretar * como "uma ou mais vezes". O * permite zero ou mais; o + exige uma ou mais.
- Ignorar os parênteses em expressões regulares.
- Confundir variáveis com terminais em uma gramática, ou tratar formas intermediárias da derivação como palavras da linguagem.

20. Aplicações em Computação

Linguagens formais aparecem em diversas áreas da Computação. Compiladores utilizam padrões para reconhecer tokens; validadores podem usar expressões regulares para conferir formatos; protocolos e linguagens de programação são descritos por regras formais; autômatos reconhecem conjuntos de palavras; e a análise de linguagens permite estudar quais problemas podem ou não ser resolvidos por determinados modelos computacionais.

As expressões regulares estudadas nesta unidade formam a ponte direta para a Unidade 3, em que as linguagens regulares passam a ser reconhecidas por autômatos finitos.

21. Questões de fixação

Questão 1. Considere $\Sigma = \{0,1\}$. Dê três palavras de comprimento 4 pertencentes a Σ^* e determine o comprimento de cada uma.
Questão 2. Explique a diferença entre Σ, Σ^*, ε e $\varnothing$.
Questão 3. Se $L = \{a,b\}$, escreva L^2 e dê cinco exemplos de palavras pertencentes a L^*.
Questão 4. Considere $L_1 = \{a,ab,b\}$ e $L_2 = \{b,ba\}$. Calcule $L_1 \cup L_2$, $L_1 \cap L_2$ e $L_1 - L_2$.
Questão 5. Explique por que ε sempre pertence a L^*.
Questão 6. Para $\Sigma=\{a,b,0,1\}$, determine se as palavras 01, a10, bb0110, aab e 1010 pertencem à linguagem descrita por (a|b)*(01|10)+.
Questão 7. Identifique V, Σ, P e S na gramática $G = (\{S, A\}, \{0, 1\}, P, S)$, com P: $S \to 0A \mid \varepsilon$ e $A \to 1S$.
Questão 8. Mostre uma derivação da palavra aaabbb na gramática $S \to aSb \mid \varepsilon$ e explique a diferença entre variável e símbolo terminal.

22. Síntese da unidade

Um alfabeto Σ é um conjunto finito e não vazio de símbolos. Palavras são sequências finitas desses símbolos, e seu comprimento é indicado por $|w|$. A palavra vazia ε possui comprimento zero. Uma linguagem é um conjunto de palavras sobre um alfabeto e, formalmente, é um subconjunto de Σ^*.
Uma gramática formal $G = (V, \Sigma, P, S)$ descreve uma linguagem por meio de regras de produção: partindo do símbolo inicial S, as derivações substituem variáveis até restarem apenas terminais, e o conjunto das palavras assim obtidas é a linguagem L(G).
Palavras e linguagens admitem operações como concatenação e potência. Entre linguagens, também utilizamos união, interseção, diferença, complemento, fecho de Kleene e fecho positivo. A Hierarquia de Chomsky organiza linguagens por seu poder de expressão, e as linguagens regulares ocupam o nível Tipo 3.
Expressões regulares permitem descrever linguagens regulares de forma compacta. Seus principais operadores são alternativa (|), concatenação, fecho de Kleene (*), fecho positivo (+) e agrupamento por parênteses.

UNIDADE 3 - Autômatos Finitos

1. Introdução

Autômatos finitos são modelos matemáticos utilizados para representar sistemas com uma quantidade finita de estados. Eles recebem uma sequência de símbolos como entrada, processam essa sequência símbolo por símbolo e, ao final, determinam se a palavra deve ser aceita ou rejeitada.

Na Teoria da Computação, autômatos finitos estão diretamente relacionados às linguagens regulares. Eles ajudam a responder uma pergunta central: dada uma palavra w, é possível decidir, por meio de um mecanismo com memória finita, se w pertence a uma linguagem L?

Além do interesse teórico, ideias semelhantes aparecem em analisadores léxicos, validação de padrões, protocolos, circuitos de controle, interfaces e máquinas de estados utilizados em software.

2. O que é um autômato finito?

Um autômato finito pode ser imaginado como uma máquina que possui um estado atual. Ao ler cada símbolo da entrada, ela consulta uma regra de transição e muda, ou não, de estado. Como o conjunto de estados é finito, a máquina possui apenas uma quantidade limitada de informação sobre aquilo que já leu.

Ao final da leitura, a palavra é aceita se toda a entrada tiver sido consumida e o autômato estiver em um estado de aceitação.

> *Essa regra é fundamental: chegar a um estado final antes de terminar a palavra não significa que a entrada já foi aceita. A leitura precisa terminar.*

3. Definição formal: a 5-upla

$M = (Q, \Sigma, \delta, q_0, F)$

Um autômato finito é formalmente descrito por cinco componentes:

Componente	Significado
Q	Conjunto finito de estados.
Σ	Alfabeto de entrada: conjunto finito de símbolos que podem aparecer na palavra.
δ	Função de transição: determina a mudança de estado durante a leitura.
q_0	Estado inicial, com $q_0 \in Q$.
F	Conjunto de estados finais ou de aceitação, com $F \subseteq Q$.

No caso de um AFD, a função de transição tem a forma:

$\delta : Q \times \Sigma \to Q$

Isso significa que, para cada combinação formada por um estado e um símbolo do alfabeto, existe exatamente um próximo estado.

4. Como interpretar a função de transição

A notação $\delta(q, a) = p$ pode ser lida da seguinte maneira: se o autômato estiver no estado q e ler o símbolo a, ele passa para o estado p.

$\delta(q_0, a) = q_1$

Portanto, q_0 é o estado de origem, a é o símbolo consumido da entrada e q_1 é o próximo estado.

É importante perceber que a transição consome o símbolo lido. Depois disso, a máquina continua com o restante da palavra.

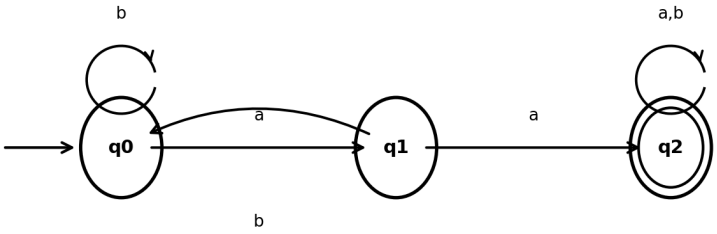
5. Exemplo baseado nos estados do slide

O conteúdo-base apresenta $Q = \{q_0, q_1, q_2\}$, $\Sigma = \{a,b\}$, q_0 como estado inicial, $F = \{q_2\}$ e algumas transições, como $\delta(q_0,a)=q_1$, $\delta(q_1,a)=q_2$ e $\delta(q_1,b)=q_1$.

Essas transições ajudam a ilustrar a notação, mas, tomadas isoladamente, não formam a tabela completa de um AFD, porque faltam regras para algumas combinações estado-símbolo. Em um AFD formalmente completo, deve existir exatamente uma transição para cada par em Q × Σ.

Para fins didáticos, podemos construir um exemplo completo usando os mesmos três estados. O autômato abaixo reconhece palavras sobre {a,b} que contêm a sequência 'aa' em algum ponto.

Q = {q_0, q_1, q_2} Σ = {a,b} F = {q_2}



Exemplo completo: reconhece palavras sobre {a,b} que contêm 'aa'.

5.1 Tabela de transição do exemplo

Estado atual	Lê a	Lê b	Observação
q_0	q_1	q_0	Ainda não encontramos 'aa'.
q_1	q_2	q_0	Já vimos um 'a'; outro 'a' completa 'aa'.
q_2	q_2	q_2	A sequência 'aa' já ocorreu; continuamos aceitando.

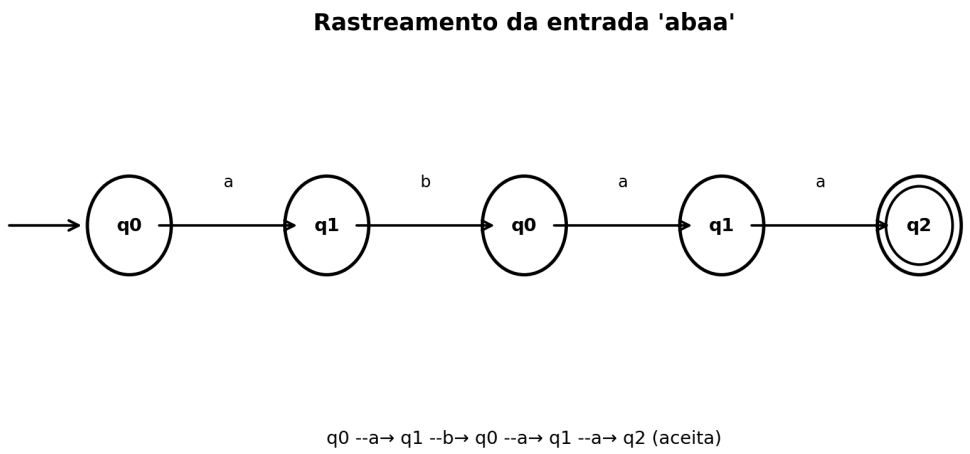
O estado q_2 é final. Depois que a sequência 'aa' é encontrada, qualquer restante da entrada mantém a máquina em q_2.

6. Executando um AFD passo a passo

Para verificar uma palavra, começamos em q_0 e processamos os símbolos na ordem em que aparecem.

Considere a entrada:

w = abaa



A execução é:

q_0 --a→ q_1 --b→ q_0 --a→ q_1 --a→ q_2

Toda a entrada foi consumida e o estado final é q_2, que pertence a F. Portanto, a palavra abaa é aceita.

Já a palavra abab terminaria em q_0 e seria rejeitada, pois q_0 não pertence a F.

7. Configuração e estado atual

Durante o processamento, duas informações são especialmente importantes: o estado atual e a parte da entrada que ainda não foi consumida. Uma forma informal de representar uma configuração é escrever o estado seguido da entrada restante.

(q_0, abaa) → (q_1, baa) → (q_0, aa) → (q_1, a) → (q_2, ε)

Ao chegar a ε, não resta símbolo para ler. Como o estado atual é q_2 e q_2 é final, a entrada é aceita.

8. AFD — Autômato Finito Determinístico

No AFD, para cada estado e para cada símbolo do alfabeto existe exatamente uma transição possível. Isso significa que, durante a execução, nunca há dúvida sobre qual caminho seguir.

$\delta : Q \times \Sigma \to Q$

Se o autômato está no estado q_0 e lê a, existe um único próximo estado determinado por $\delta(q_0,a)$. Por isso o comportamento é chamado determinístico.

9. AFND — Autômato Finito Não Determinístico

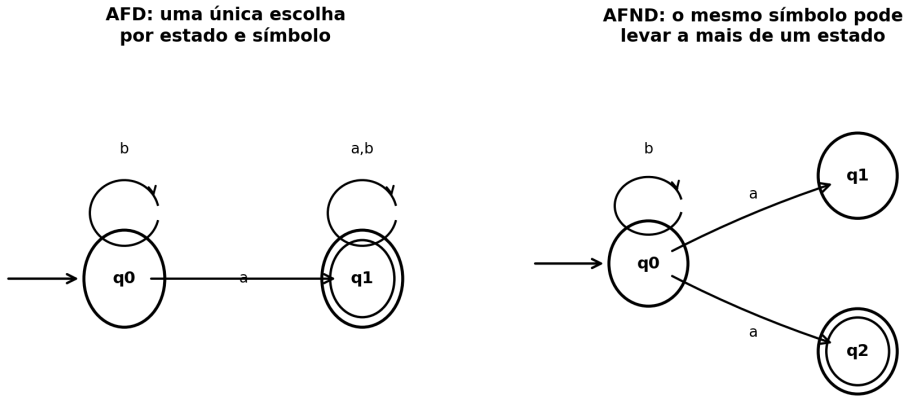
No AFND, uma mesma combinação de estado e símbolo pode permitir nenhuma, uma ou várias transições. Assim, uma entrada pode gerar vários caminhos possíveis de execução.

Uma maneira formal comum de escrever a função de transição de um AFND é:

$\delta : Q \times \Sigma \to P(Q)$

Aqui, P(Q) representa o conjunto das partes de Q. O resultado da função pode ser um conjunto de estados possíveis.

Algumas definições de AFND também permitem transições ε, nas quais a máquina muda de estado sem consumir símbolo da entrada. O slide-base desta unidade enfatiza principalmente a possibilidade de múltiplos caminhos para o mesmo símbolo.



10. Critério de aceitação no AFND

Uma palavra é aceita por um AFND se existir pelo menos um caminho de execução que consuma toda a entrada e termine em um estado de aceitação.

Isso significa que caminhos que travam ou terminam em estados não finais não impedem a aceitação, desde que outro caminho possível aceite a palavra.

Em termos intuitivos, o AFND pode explorar possibilidades. Para aceitar, basta que uma delas funcione.

11. AFD versus AFND

Característica	AFD	AFND
Transições por estado e símbolo	Exatamente uma	Zero, uma ou várias
Caminho de execução	Único	Pode haver vários
Critério de aceitação	O único caminho termina em estado final	Pelo menos um caminho termina em estado final
Poder de reconhecimento	Linguagens regulares	Linguagens regulares
Conversão	Já determinístico	Pode ser convertido em AFD equivalente

AFD e AFND possuem o mesmo poder de reconhecimento: ambos reconhecem exatamente as linguagens regulares. A diferença está na forma de representar e executar o reconhecimento, não na classe de linguagens que podem reconhecer.

12. Conversão de AFND para AFD

Um AFND pode ser convertido em um AFD equivalente por uma técnica conhecida como construção dos subconjuntos. A ideia é que um estado do AFD resultante represente um conjunto de estados que o AFND poderia ocupar simultaneamente.

Por exemplo, se após ler um símbolo o AFND poderia estar em q_1 ou q_2, o AFD equivalente pode utilizar um estado composto $\{q_1,q_2\}$.

Como um conjunto com n estados possui até 2^n subconjuntos, a conversão pode aumentar bastante o número de estados. Esse crescimento explica por que uma representação não determinística pode ser mais compacta mesmo sem possuir maior poder de reconhecimento.

13. Linguagem reconhecida por um autômato

A linguagem reconhecida por um autômato M é o conjunto de todas as palavras que M aceita. Essa linguagem costuma ser indicada por L(M).

$L(M) = \{ w \in \Sigma^* \mid M \text{ aceita } w \}$

No exemplo completo apresentado anteriormente, L(M) contém exatamente as palavras sobre {a,b} que possuem 'aa' como substring. Exemplos incluem aa, aab, baa, abaa e bbaabb.

14. Estado final não significa parada imediata

Uma confusão frequente é imaginar que a máquina deve parar assim que alcança um estado final. Em um autômato finito padrão, isso não é necessariamente verdade. O estado final indica aceitação somente quando toda a palavra foi consumida.

Uma máquina pode entrar em um estado final no meio da palavra e depois sair dele. Nesse caso, o que importa é o estado em que termina após a leitura completa.

No exemplo usado neste documento, q_2 possui laços para a e b, então, depois de entrar em q_2, a máquina permanece nele. Essa é uma escolha do exemplo, não uma regra geral.

15. O que um AFD consegue memorizar?

A memória de um AFD está embutida nos próprios estados. Cada estado pode representar uma quantidade finita de informação sobre o prefixo já lido.

Por exemplo, um AFD pode lembrar se uma quantidade é par ou ímpar usando dois estados. Pode lembrar os últimos dois símbolos usando uma quantidade finita de estados. Também pode reconhecer se determinada substring já apareceu.

O problema surge quando a informação necessária pode crescer sem limite. Como o número de estados é fixo, não há como representar arbitrariamente muitos valores distintos.

16. Principais limitações de um AFD

- Possui memória finita: a quantidade de estados é fixa.
- Não mantém um contador arbitrariamente grande.
- Não compara duas quantidades sem limite, como exigir o mesmo número de a e b para qualquer tamanho de entrada.
- Não armazena uma palavra arbitrariamente longa para compará-la posteriormente com outra parte da entrada.
- Reconhece linguagens regulares, mas existem linguagens que não são regulares.

17. Exemplo clássico de limitação: L = {$a^n b^n$ | n ≥ 0}

L = { $a^n b^n$ | n ≥ 0 }

Essa linguagem contém:

ε, ab, aabb, aaabbb, aaaabbbb, ...

Para que uma palavra pertença a L, três condições devem ser respeitadas:

- A palavra começa com zero ou mais símbolos a.
- Depois aparecem símbolos b.
- A quantidade de b deve ser exatamente igual à quantidade de a.

Por que um AFD não reconhece L = {$a^n b^n$ | n ≥ 0}?

ab	1 a	1 b	✓ iguais
aabb	2 a	2 b	✓ iguais
aaabbb	3 a	3 b	✓ iguais
$a^n b^n$	n a	n b	✓ iguais

Para qualquer n, seria necessário lembrar quantos 'a' foram lidos e comparar com os 'b'.
Como n não tem limite, uma quantidade fixa de estados não oferece memória suficiente.

Para reconhecer corretamente todas as palavras dessa linguagem, uma máquina precisaria guardar a quantidade exata de a lidos para compará-la com a quantidade de b que aparecerá depois.

Como n pode ser arbitrariamente grande, não existe uma quantidade fixa de estados capaz de armazenar todos os valores possíveis de n. Por isso, nenhum AFD reconhece essa linguagem.

18. Intuição do argumento dos estados finitos

Suponha que um AFD tenha apenas k estados. Ao ler uma sequência suficientemente longa de símbolos a, com comprimento maior que k, o autômato obrigatoriamente repetirá algum estado. Isso acontece porque existem mais posições de leitura do que estados disponíveis.

Quando um estado se repete, surge um ciclo. A máquina deixa de distinguir certas quantidades diferentes de a, pois passa pelo mesmo estado após comprimentos distintos. Dessa forma, não consegue posteriormente exigir uma quantidade exatamente correspondente de b para todas as entradas possíveis.

Essa intuição é formalizada pelo Lema do Bombeamento para linguagens regulares, assunto que pode ser estudado posteriormente.

19. Diferença entre erro de ordem e erro de quantidade

A linguagem $a^n b^n$ exige tanto uma ordem específica quanto igualdade de quantidades. Um AFD consegue verificar facilmente a ordem geral 'todos os a antes de todos os b'. O problema central é comparar as quantidades sem limite.

Por exemplo, a linguagem a*b* é regular: ela aceita qualquer quantidade de a seguida de qualquer quantidade de b. Já $a^n b^n$ adiciona a exigência de igualdade entre as duas quantidades, o que ultrapassa a memória finita de um AFD.

20. Erros frequentes

- Achar que basta chegar a um estado final antes do fim da entrada. A palavra só é aceita após consumir toda a entrada.
- Esquecer que, em um AFD, cada par estado-símbolo deve possuir exatamente uma transição.
- Confundir ausência de transição em um desenho parcial com comportamento formal de um AFD completo.
- Pensar que AFND reconhece mais linguagens que AFD. Os dois reconhecem a mesma classe: as linguagens regulares.
- Pensar que um AFD não consegue contar nada. Ele consegue guardar informações finitas, como paridade; o que não consegue é manter contagens arbitrariamente grandes.
- Achar que a limitação de $a^n b^n$ ocorre porque há muitos símbolos diferentes. O problema é a necessidade de comparar uma quantidade sem limite.

21. Aplicações em Computação

Autômatos finitos são usados como modelo para vários mecanismos computacionais. Compiladores podem utilizar autômatos para reconhecer tokens; expressões regulares podem ser convertidas em autômatos; protocolos e interfaces podem ser modelados por máquinas de estados; sistemas embarcados usam estados para representar modos de operação; e validadores de padrões podem ser construídos a partir de linguagens regulares.

A principal vantagem do modelo é sua simplicidade: como existe apenas uma quantidade finita de estados, seu comportamento pode ser analisado,

representado por tabelas e diagramas e implementado de maneira eficiente.

22. Questões de fixação

Questão 1. Explique, com suas palavras, o papel de cada componente da 5-upla $M = (Q, \Sigma, \delta, q_0, F)$.

Questão 2. Qual é a diferença essencial entre AFD e AFND? Essa diferença altera a classe de linguagens reconhecidas?

Questão 3. No AFD do exemplo que reconhece palavras contendo 'aa', determine se as palavras bbb, baa, abab, aaaa e baba são aceitas.

Questão 4. Explique por que uma palavra não pode ser considerada aceita apenas porque o autômato passou por um estado final durante a leitura.

Questão 5. Por que a linguagem $\{a^n b^n \mid n \geq 0\}$ não pode ser reconhecida por um AFD?

Questão 6. Dê um exemplo de uma informação finita que um AFD consegue memorizar e de uma informação ilimitada que ele não consegue memorizar.

23. Síntese da unidade

Um autômato finito é um modelo matemático com um conjunto finito de estados que processa palavras símbolo por símbolo. Sua definição formal é $M = (Q, \Sigma, \delta, q_0, F)$. No AFD, a função δ fornece exatamente um próximo estado para cada combinação de estado e símbolo.

No AFND, podem existir várias possibilidades de transição. Mesmo assim, AFD e AFND reconhecem exatamente as linguagens regulares, e todo AFND pode ser convertido em um AFD equivalente.

A principal limitação dos autômatos finitos é a ausência de memória ilimitada. Por isso, eles não conseguem reconhecer linguagens que exigem armazenar e comparar quantidades arbitrariamente grandes, como $L = \{a^n b^n \mid n \geq 0\}$. Essa limitação motiva o estudo de modelos mais poderosos, como os autômatos de pilha da próxima unidade.

UNIDADE 4 - Autômatos de Pilha

1. Introdução

Um autômato de pilha é um modelo computacional semelhante a um autômato finito, mas que possui uma estrutura adicional de memória: a pilha. Essa memória permite armazenar símbolos e recuperá-los segundo a regra LIFO (Last In, First Out): o último símbolo inserido é o primeiro a ser removido.

O autômato finito guarda apenas o estado corrente, e o número de estados é fixo. A pilha, por não ter tamanho limitado, permite ao autômato registrar informações que um autômato finito não consegue reter, como a quantidade de símbolos já lidos.

2. Definição

$M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$

Componente	Descrição
Q	conjunto finito de estados
Σ	alfabeto de entrada
Γ	alfabeto da pilha
$q_0 \in Q$	estado inicial
$Z_0 \in \Gamma$	símbolo inicial da pilha (marcador de fundo)
δ	função de transição
$F \subseteq Q$	conjunto de estados finais

O símbolo Z_0 é colocado na pilha no início da computação e costuma funcionar como marcador de fundo. A função δ determina, a partir do estado atual, do símbolo lido e do símbolo no topo da pilha, qual será o próximo estado e o que será feito com o topo da pilha.

Esquema do modelo

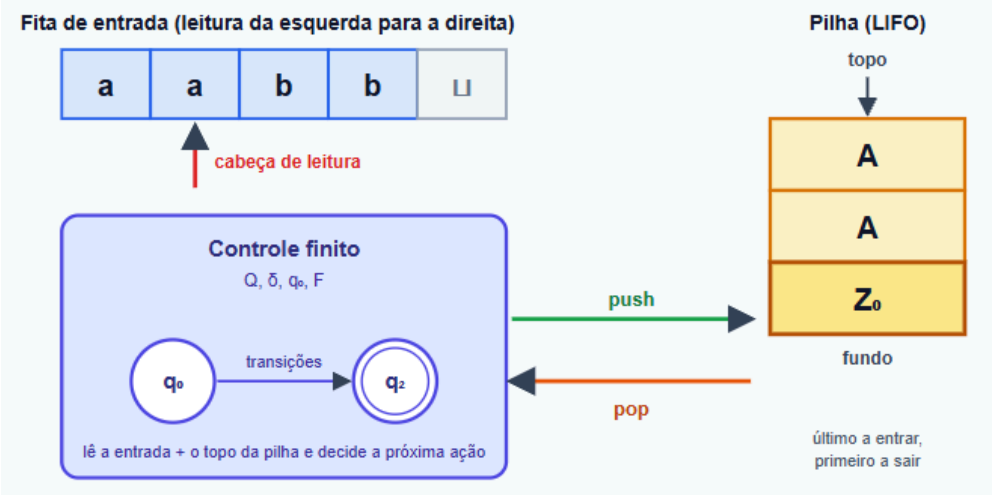


Figura 1. Esquema do autômato de pilha: fita de entrada, controle finito e pilha.

3. Transições do autômato de pilha

Uma transição pode ser representada informalmente assim:

entrada, topo → substituição

Exemplo: **b, A → ε**. Essa transição significa:

- ler o símbolo *b*;
- verificar se o topo é *A*;
- consumir *b* da entrada;
- remover *A* da pilha.

3.1 O que a substituição pode representar

Operação	Efeito	Exemplo
Push	acrescenta símbolos ao topo	a, Z_0 → AZ_0
Pop	remove o símbolo do topo	b, A → ε
Manter	preserva o topo	ε, A → A

A forma exata de escrever as transições varia conforme a definição formal utilizada. Nesta unidade adota-se a notação informal acima, em que a cadeia à direita da seta substitui o símbolo do topo. Quando a entrada é ε, a transição ocorre sem consumir símbolo da palavra.

3.2 Exemplo: reconhecimento de $a^n b^n$

Considere o autômato da Figura 2, com estados q_0, q_1, q_2 e q_3 (q_3 final). A figura utiliza a notação entrada, topo; substituição, com λ no lugar de ε e Z no lugar de Z_0. As grafias são equivalentes.

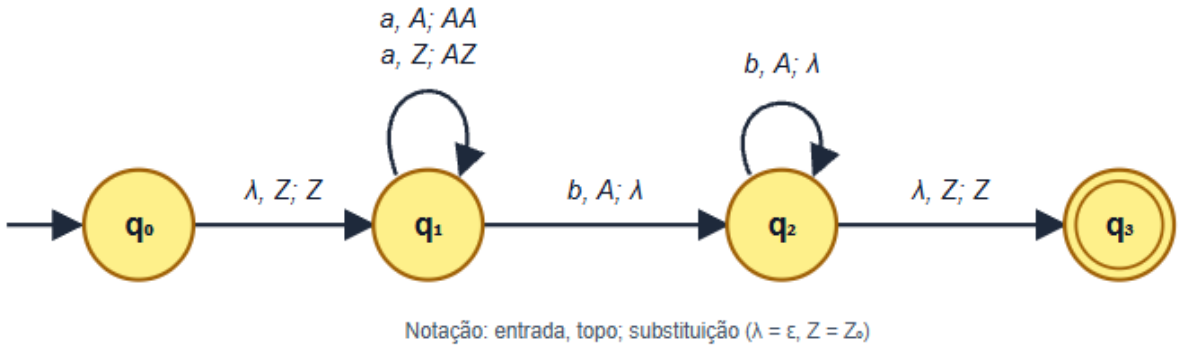


Figura 2. Autômato de pilha para $a^n b^n$ ($n \geq 1$).

Transição	Origem → destino	Efeito
λ, Z; Z	$q_0 \rightarrow q_1$	não lê símbolo e mantém Z no topo (início da computação)
a, Z; AZ	$q_1 \rightarrow q_1$	primeiro a: empilha A sobre Z
a, A; AA	$q_1 \rightarrow q_1$	demais a: empilha mais um A
b, A; λ	$q_1 \rightarrow q_2$	primeiro b: remove um A
b, A; λ	$q_2 \rightarrow q_2$	demais b: remove um A
λ, Z; Z	$q_2 \rightarrow q_3$	pilha reduzida a Z: vai ao estado final q_3

Observação: o autômato da figura reconhece $a^n b^n$ com $n \geq 1$. Para incluir também a palavra vazia ($n = 0$), como na definição da linguagem, basta acrescentar uma transição λ, Z; Z de q_1 para q_3.

3.3 Evolução da pilha ao processar aabb:

início	lê a	lê a	lê b	lê b
·	·	**A**	·	·
·	**A**	**A**	**A**	·
Z0	**Z0**	**Z0**	**Z0**	**Z0**

Ao final da entrada, a pilha contém apenas Z_0, o autômato passa ao estado final q_3 e a palavra é aceita. Para aab restaria um A na pilha, e para abb o segundo b encontraria Z_0 no topo, sem transição definida em q_2. Em ambos os casos a palavra é rejeitada.

4. Linguagens reconhecidas por autômatos de pilha

Autômatos de pilha não determinísticos reconhecem exatamente as linguagens livres de contexto, isto é, as linguagens geradas por gramáticas livres de contexto.

4.1 Exemplos

L = { $a^n b^n$ | n ≥ 0 }: mesma quantidade de a e b, nessa ordem. O autômato da seção 2.2 empilha um símbolo por a e desempilha um por b.
Linguagens de parênteses balanceados: cada abertura empilha um símbolo e cada fechamento remove um (autômato completo na seção 3.2). Palavras com estruturas aninhadas, como (()()). A altura da pilha indica a profundidade de aninhamento:

símbolo:	(	(	)	(	)	)	
altura:	1	2	1	2	1	0	balanceada

Palavras geradas por gramáticas livres de contexto. Por exemplo, a gramática S → aSb | **ε** gera **ε**, ab, aabb, aaabbb, ..., que é a linguagem $a^n b^n$.

4.2 Parênteses balanceados: autômato completo

Uma palavra sobre {(,)} é balanceada quando cada fechamento corresponde a uma abertura anterior e nenhuma abertura fica sem fechamento. O autômato usa o símbolo P para registrar cada abertura pendente.

Transição	**Efeito**
(, Z; PZ	primeira abertura: empilha P sobre Z
(, P; PP	nova abertura: empilha mais um P
), P; λ	fechamento: remove um P
λ, Z; Z	pilha reduzida a Z: vai ao estado final q_1

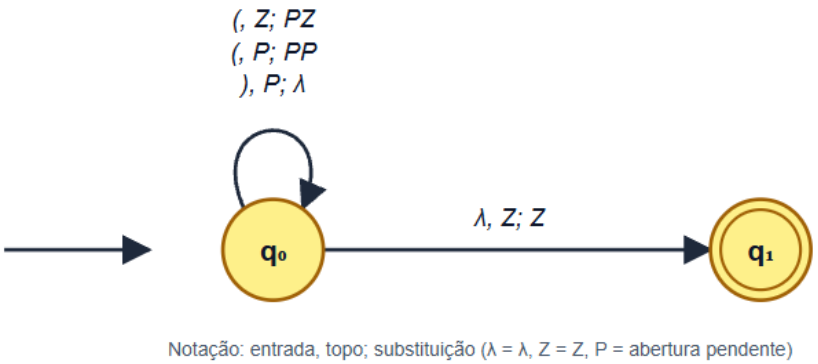


Figura 3. Autômato de pilha para parênteses balanceados.

Palavra aceita: (()()). A pilha cresce a cada abertura e diminui a cada fechamento.

Passo	Estado	Entrada restante	Pilha (topo à esquerda)	Transição aplicada
0	q_0	(()())	Z	início
1	q_0	()())	PZ	(, Z; PZ
2	q_0	)())	PPZ	(, P; PP
3	q_0	())	PZ	), P; λ
4	q_0	))	PPZ	(, P; PP
5	q_0	)	PZ	), P; λ
6	q_0	λ	Z	), P; λ
7	q_1	λ	Z	λ, Z; Z

A entrada terminou, a pilha voltou a Z e o estado é final: a palavra é aceita.

Palavra rejeitada: ((). Falta um fechamento.

Passo	Estado	Entrada restante	Pilha (topo à esquerda)	Transição aplicada
0	q_0	(()	Z	início
1	q_0	()	PZ	(, Z; PZ
2	q_0	)	PPZ	(, P; PP
3	q_0	λ	PZ	), P; λ

A entrada acabou, mas o topo da pilha é P, e a única transição sem leitura, λ, Z; Z, exige Z no topo. Como o autômato não chega ao estado final, a palavra é rejeitada. A pilha não vazia indica quantas aberturas ficaram sem fechamento (aqui, uma).

Outro caso de rejeição: ()). Após () a pilha volta a Z. O segundo) encontra Z no topo e não há transição), Z. A computação é interrompida, o que indica um fechamento sem abertura correspondente.

4.3 O papel da pilha

A pilha permite lidar com certas dependências que exigem memória não limitada, como comparar a quantidade de símbolos de duas partes da entrada. É o que ocorre em $a^n b^n$: a quantidade de b depende da quantidade de a lida anteriormente, e a pilha guarda essa informação.

4.4 Limitação

Um único autômato de pilha não determinístico não reconhece todas as linguagens possíveis. Por exemplo, a linguagem

$L = \{ a^n b^n c^n \mid n \geq 0 \}$

Não é livre de contexto e não pode ser reconhecida por um autômato de pilha comum.
Porque a pilha não basta. Para reconhecer $a^n b^n c^n$ seria preciso verificar duas igualdades: a quantidade de a igual à de b e a quantidade de b igual à de c. A pilha registra a quantidade de a e, para compará-la com a de b, o autômato precisa desempilhar. Ao término dos b, a pilha está vazia e a informação sobre n foi perdida. Nada resta para comparar com os c.

O esquema abaixo mostra a pilha ao processar aabbcc:

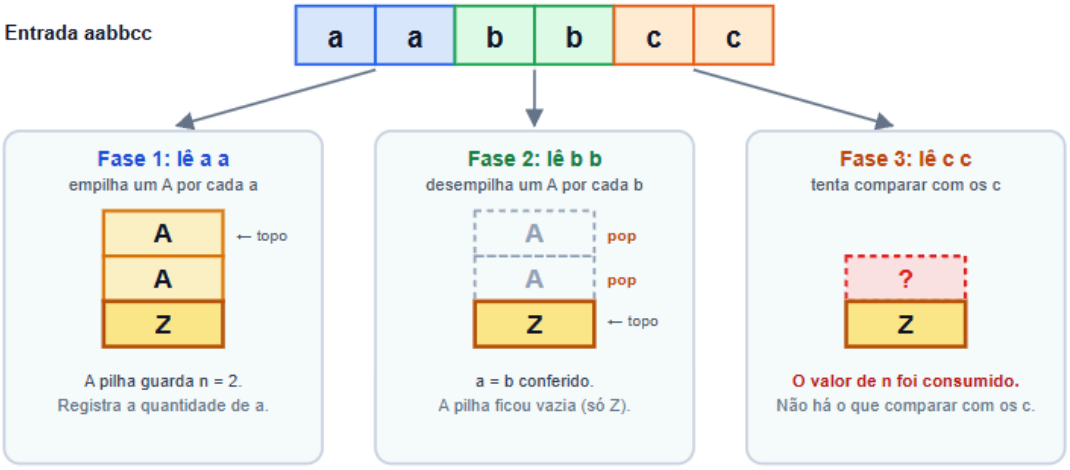


Figura 4. Conteúdo da pilha ao processar aabbcc.

Alternativas que não resolvem o problema:

- Não desempilhar ao ler os b: a pilha continuaria com n símbolos para os c, mas então o autômato não teria conferido a igualdade entre a e b.
- Guardar n nos estados: o conjunto de estados é finito e n é ilimitado, de modo que nenhuma quantidade fixa de estados basta.
- Usar uma segunda pilha: resolveria o problema, mas o modelo deixaria de ser o autômato de pilha considerado nesta unidade, que possui uma única pilha.

Em resumo, com uma única pilha é possível confrontar duas quantidades, mas não três. A demonstração formal desse resultado é feita com o lema do bombeamento para linguagens livres de contexto.

5. Como resolver exercícios de autômatos de pilha

5.1 Roteiro de resolução

A construção de um autômato de pilha para uma linguagem dada pode seguir a sequência abaixo.

Identificar a dependência. Determinar qual informação precisa ser lembrada sem limite (por exemplo, a quantidade de a lidos) e com que parte da entrada ela será comparada.
Definir a estratégia da pilha. Estabelecer o que será empilhado, em que momento e em que momento será removido.
Definir os estados. Em geral, um estado por fase da leitura (por exemplo, fase dos a e fase dos b), além de um estado final.

Escrever as transições no formato entrada, topo → substituição, cobrindo o primeiro símbolo (topo Z_0), os demais símbolos, a remoção e a transição **ε** para o estado final.
Simular palavras. Testar palavras aceitas e rejeitadas, incluindo a palavra vazia, o excesso de cada símbolo e a ordem incorreta.
Conferir as transições ausentes. Quando não existe transição aplicável, a computação é interrompida e a palavra é rejeitada. Não é necessário um estado de rejeição.

5.2 Exercício resolvido: L = { $a^n b^{2n}$ | n ≥ 0 }

Enunciado. Construir um autômato de pilha que reconheça as palavras formadas por n símbolos a seguidos de exatamente 2n símbolos b.

Dependência. A quantidade de b é o dobro da quantidade de a.

Estratégia: Empilhar dois símbolos A para cada a lido e desempilhar um A para cada b lido. Se a entrada terminar com apenas Z_0 na pilha, a quantidade de b é exatamente 2n.

Estados. q_0 (leitura dos a), q_1 (leitura dos b) e q_2 (final).

Transições

Transição	**Origem → destino**	**Efeito**
a, Z_0 → AAZ_0	q_0 → q_0	primeiro a: empilha dois A sobre Z_0
a, A → AAA	q_0 → q_0	demais a: troca o topo A por três A, isto é, acrescenta dois A
b, A → ε	q_0 → q_1	primeiro b: remove um A
b, A → ε	q_1 → q_1	demais b: remove um A
ε, Z_0 → Z_0	q_0 → q_2	aceita a palavra vazia (n = 0)
ε, Z_0 → Z_0	q_1 → q_2	pilha reduzida a Z_0: aceita

Simulação com aabbbb (aceita)

Passo	Estado	Entrada restante	Pilha (topo à esquerda)	Transição aplicada
0	q_0	aabbbb	Z_0	início
1	q_0	abbbb	AAZ_0	a, $Z_0 \to AAZ_0$
2	q_0	bbbb	$AAAAZ_0$	a, $A \to AAA$
3	q_1	bbb	$AAAZ_0$	b, $A \to \varepsilon$
4	q_1	bb	AAZ_0	b, $A \to \varepsilon$
5	q_1	b	AZ_0	b, $A \to \varepsilon$
6	q_1	ε	Z_0	b, $A \to \varepsilon$
7	q_2	ε	Z_0	ε, $Z_0 \to Z_0$

Palavras rejeitadas:

aabbb (falta um b): a entrada termina em q_1 com AZ_0 na pilha. Como o topo é A, a transição $\varepsilon, Z_0 \to Z_0$ não se aplica, e a palavra é rejeitada.

abbb (excesso de b): após os dois primeiros b, a pilha reduz-se a Z_0. O terceiro b encontra Z_0 no topo, e não há transição definida. A computação é interrompida e a palavra é rejeitada.

5.3 Como decidir se uma linguagem é reconhecida por AP

Regra prática:
Um autômato de pilha (AP) consegue comparar até duas quantidades.
Se precisar comparar três ou mais, geralmente não é reconhecível.

Resumo das linguagens

Linguagem	Reconhece?	Ideia principal
$a^n b^n$	Sim	empilha a, desempilha em b
$a^n b^{2n}$	Sim	empilha 2 por a
$a^n b^m c^n$	Sim	compara a com c
$a^n b^m$, com $m \geq n$	Sim	b excedente não interfere
$a^n b^n c^n$	Não	exige 3 comparações

Resumo mental: AP = “compara dois grupos” e 3 grupos → não dá

*Observação: prova formal usa **lema do bombeamento (CFL)***

Esse critério é uma orientação para a construção. A demonstração formal de que uma linguagem não é livre de contexto utiliza o lema do bombeamento para linguagens livres de contexto.

6. Erros frequentes

- Σ ≠ Γ → entrada ≠ pilha
- Não usar Z_0 → perde controle do fim
- Empilhar na ordem errada → topo é sempre à esquerda
- ε ≠ esvaziar tudo → remove só o topo
- Transição ε não consome entrada
- Ignorar palavra vazia (ε) quando permitido
- Aceitar sem olhar pilha → errado (topo deve ser Z_0)
- Criar “estado de erro” → rejeição é falta de transição
- Confundir AP determinístico vs não determinístico
- Achar que $a^n b^n c^n$ funciona → não funciona

7. Aplicações em Computação

O autômato de pilha (AP) é usado quando é necessário controlar estruturas aninhadas (memória tipo LIFO).

Principais aplicações:

- Compiladores: análise sintática (gramáticas livres de contexto)
- Editores de código: verificação de parênteses/chaves balanceados
- HTML/XML/JSON: validação de tags aninhadas
- Expressões matemáticas: uso de pilha para cálculo
- Recursão: controle da pilha de chamadas

Exemplo ({[()]}):
Empilha ao abrir e desempilha ao fechar corretamente → aceita
Se houver erro de correspondência ({[)]}) → rejeita

8. Questões de fixação

Questão 1. Escreva a definição formal de um autômato de pilha e explique a função de cada componente, destacando o papel de Γ e de Z_0.

Questão 2. A pilha contém apenas Z_0. Determine o conteúdo da pilha (topo à esquerda) após a sequência de operações: push A, push B, pop, push C, push A, pop.

Questão 3. Interprete as transições b, A → ε, a, Z_0 → AZ_0 e ε, A → A, indicando, para cada uma, se há consumo de entrada e qual operação ocorre na pilha (push, pop ou manter).

Questão 4. Considere o autômato da Figura 2, que utiliza a notação das figuras (λ = ε e Z = Z_0). Determine se ele aceita as palavras aaabbb, aab, abb, ba e a palavra vazia, justificando cada resposta.

Questão 5. No autômato da Figura 3, simule o processamento das palavras (()) e)(, indicando o conteúdo da pilha em cada passo e o resultado.

Questão 6. Indique quais das linguagens abaixo podem ser reconhecidas por um autômato de pilha e justifique.

(a) $\{ a^n b^n \mid n \geq 0 \}$

(b) $\{ a^n b^m c^n \mid n, m \geq 0 \}$

(c) $\{ a^n b^n c^n \mid n \geq 0 \}$

(d) $\{ a^n b^m \mid m \geq n \geq 0 \}$

UNIDADE 5 - Máquinas de Turing

1. Introdução

A Máquina de Turing (MT) é um modelo matemático de computação capaz de representar algoritmos por meio de uma fita, uma cabeça de leitura e escrita e um conjunto de estados. Foi proposta por Alan Turing em 1936, no artigo em que ele respondeu negativamente ao *Entscheidungsproblem* (problema da decisão) de Hilbert, e formaliza a noção intuitiva de "procedimento efetivo": uma sequência finita e mecânica de passos que transforma uma entrada em um resultado (TURING, 1936).

Os modelos vistos nas unidades anteriores têm limitações de memória. O autômato finito guarda apenas o estado corrente. O autômato de pilha acrescenta uma pilha, mas só tem acesso ao símbolo do topo. A MT elimina essas restrições: sua fita é ilimitada e pode ser lida, escrita e percorrida nos dois sentidos, o que a torna capaz de executar qualquer algoritmo (SIPSER, 2007; HOPCROFT; MOTWANI; ULLMAN, 2002).

Em cada passo, a MT pode:

- ler um símbolo da fita;
- escrever um símbolo na fita;
- mover a cabeça para a esquerda (L) ou para a direita (R);
- mudar de estado;
- continuar o processamento até aceitar, rejeitar ou executar indefinidamente.

1.1 Componentes

Componente	Descrição
Controle finito	mantém o estado atual e contém as regras de transição
Fita	armazena a entrada e pode ser usada como memória; as células fora da entrada contêm o símbolo branco (␣)

Componente	Descrição
Cabeça de leitura e escrita	lê e modifica o símbolo da posição atual e se movimenta pela fita

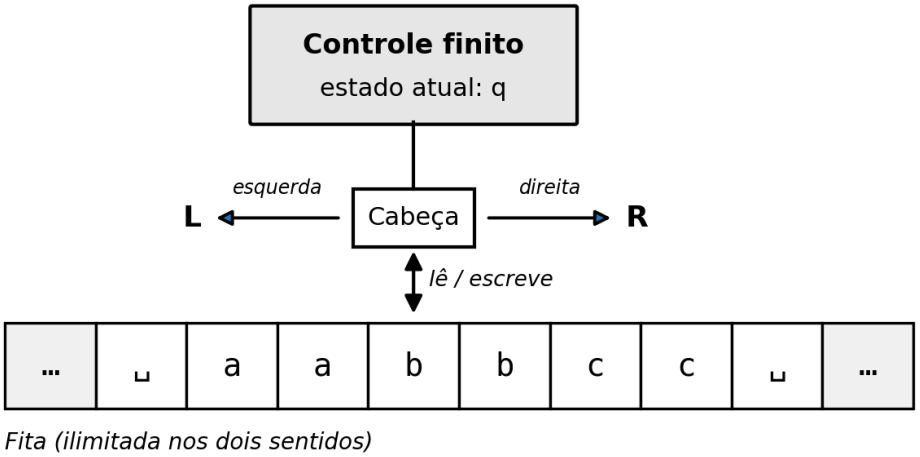


Figura 1. Esquema da Máquina de Turing: fita, cabeça de leitura e escrita e controle finito.

No modelo teórico, a fita é ilimitada, geralmente só para a direita ou em ambas as direções, conforme a convenção adotada. As duas convenções têm o mesmo poder computacional. Em Sipser (2007), por exemplo, a fita é infinita à direita e, se a cabeça tentar ir além da primeira célula à esquerda, ela permanece nessa célula.

1.2 Definição

$M = (Q, \Sigma, \Gamma, \delta, q_0, F)$

Componente	Descrição
Q	conjunto finito de estados
Σ	alfabeto de entrada (não contém o símbolo branco)
Γ	alfabeto da fita, com $\Sigma \subseteq \Gamma$ e $\sqcup \in \Gamma \setminus \Sigma$
δ	função de transição, $\delta: Q \times \Gamma \to Q \times \Gamma \times \{L, R\}$
$q_0 \in Q$	estado inicial
$F \subseteq Q$	conjunto de estados de aceitação

Observação: diferentemente do autômato de pilha, a MT não tem símbolo inicial de pilha (Z_0): a entrada já ocupa a fita, e Γ é o alfabeto da fita, que inclui o símbolo branco ␣. Muitos textos, como Sipser (2007), trocam F por dois estados de parada, q_aceita e q_rejeita; as duas formulações têm o mesmo poder computacional, e nesta unidade adota-se a definição com F.

2 Transições da Máquina de Turing

Uma transição é uma regra da função δ e depende sempre do **estado atual** e do **símbolo lido**. Ela pode ser representada informalmente assim:

símbolo lido → símbolo escrito, movimento

Exemplo: 1 → 0, R. Essa transição significa:

- se a cabeça estiver lendo 1;
- escrever 0 na posição atual;
- mover a cabeça uma posição para a direita (R = right);
- atualizar o estado conforme a regra.

Na notação formal, a mesma regra seria $\delta(q_1, 1) = (q_2, 0, R)$: em q_1, lendo 1, escreve 0, move para a direita e passa para q_2. Em um diagrama de estados, "1 → 0, R" é o rótulo do arco de q_1 para q_2.

2.1 O que uma transição pode fazer

Operação	Efeito	Exemplo
Escrever	substitui o símbolo da posição atual	1 → 0, R
Manter	reescreve o mesmo símbolo (apenas percorre a fita)	a → a, R
Mover para a direita	avança a cabeça uma célula	b → b, R
Mover para a esquerda	recua a cabeça uma célula	Y → Y, L

2.2 Configurações

Para descrever uma execução usa-se a **configuração**, um retrato instantâneo da máquina com o estado, o conteúdo da fita e a posição da cabeça. Nas tabelas deste documento, a posição da cabeça é indicada por colchetes: em X[b]c, a cabeça está sobre o b. Um passo de computação leva de uma configuração à seguinte, e uma computação começa na configuração inicial, com q_0 e a cabeça sobre o primeiro símbolo da entrada (SIPSER, 2007).

2.3 Exemplo: reconhecimento de $a^n b^n c^n$

Estratégia: A cada rodada, a máquina marca um a como X, um b como Y e um c como Z, e volta ao início. Quando não restam a's, verifica se sobraram apenas Y's e Z's até o final. Se sobrar algum b ou c sem par, ou se faltar algum símbolo, não há transição e a palavra é rejeitada.

Considere a MT com estados q_0 a q_4 e qacc (aceitação), $\Sigma = \{a, b, c\}$ e $\Gamma = \Sigma \cup \{X, Y, Z, \sqcup\}$. Qualquer situação não listada leva à rejeição.

Estado	Transições	Papel
q_0	a → X, R (q_1); Y → Y, R (q_4); ⌴ → ⌴, R (qacc)	início de rodada: marca um a; se não há mais a's, vai conferir o resto; fita vazia é aceita (n = 0)
q_1	a → a, R (q_1); Y → Y, R (q_1); b → Y, R (q_2)	procura o primeiro b não marcado e o marca
q_2	b → b, R (q_2); Z → Z, R (q_2); c → Z, L (q_3)	procura o primeiro c não marcado e o marca
q_3	a, b, Y, Z → mesmo símbolo, L (q_3); X → X, R (q_0)	volta à esquerda até o último X e reinicia
q_4	Y → Y, R (q_4); Z → Z, R (q_4); ⌴ → ⌴, R (qacc)	confere que só restam Y's e Z's

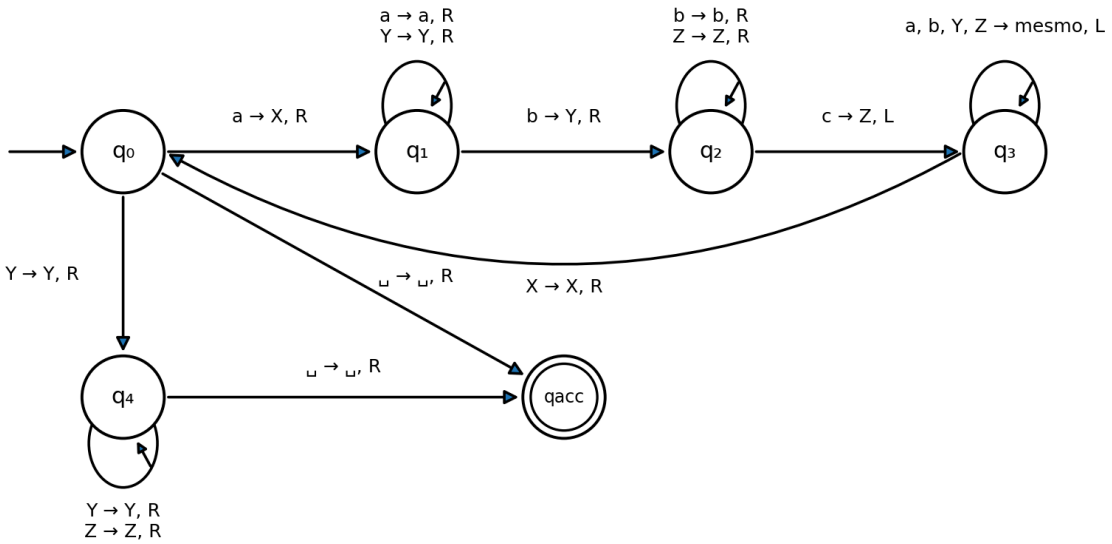


Figura 2. Diagrama de estados da MT que decide $a^n b^n c^n$ ($n \geq 0$).

Palavra aceita: abc. A fita é modificada a cada rodada e, no final, contém apenas marcas.

Passo	Estado	Fita	Transição aplicada
0	q_0	[a]bc	início
1	q_1	X[b]c	a → X, R
2	q_2	XY[c]	b → Y, R
3	q_3	X[Y]Z	c → Z, L
4	q_3	[X]YZ	Y → Y, L
5	q_0	X[Y]Z	X → X, R
6	q_4	XY[Z]	Y → Y, R
7	q_4	XYZ[␣]	Z → Z, R
8	qacc	XYZ␣[␣]	␣ → ␣, R

A máquina entrou no estado de aceitação: a palavra é aceita.

Palavra rejeitada: aabc. Falta um b para o segundo a.

Passo	Estado	Fita	Transição aplicada
0	q_0	[a]abc	início
1	q_1	X[a]bc	a → X, R
2	q_1	Xa[b]c	a → a, R

Passo	Estado	Fita	Transição aplicada
3	q_2	XaY[c]	b → Y, R
4	q_3	Xa[Y]Z	c → Z, L
5	q_3	X[a]YZ	Y → Y, L
6	q_3	[X]aYZ	a → a, L
7	q_0	X[a]YZ	X → X, R
8	q_1	XX[Y]Z	a → X, R
9	q_1	XXY[Z]	Y → Y, R

Em q_1, lendo Z, não há transição definida. A computação é interrompida sem chegar a um estado de aceitação, e a palavra é rejeitada. Como cada rodada marca um símbolo de cada tipo e a fita usada é finita, a máquina termina para qualquer entrada.

3. Aceitação, rejeição e execução infinita

Ao processar uma entrada, uma MT pode ter três comportamentos:

Comportamento	Descrição
Aceitar	a máquina entra em um estado de aceitação
Rejeitar	a máquina para sem aceitar, por atingir um estado de rejeição ou por não haver transição definida
Não parar	a máquina continua executando indefinidamente (loop)

Observação: dizer que rejeitar é "não terminar em um estado de aceitação" é ambíguo, pois também descreve o loop. Rejeitar é **parar** sem aceitar; o loop é **nunca parar**. Essa distinção sustenta toda a teoria de decidibilidade.

Um exemplo de loop: uma máquina com as regras $\delta(q, a) = (q, a, R)$ e $\delta(q, \sqcup) = (q, \sqcup, R)$ percorre a fita para a direita sem nunca parar, qualquer que seja a entrada.

3.1 Reconhecer e decidir

Conceito	Definição
Reconhecer	aceitar exatamente as palavras da linguagem; nas demais, a máquina pode rejeitar ou entrar em loop
Decidir	aceitar toda palavra da linguagem, rejeitar toda palavra que não pertence a ela e sempre parar

Toda linguagem decidível é reconhecível, mas a recíproca é falsa, como se vê na Seção 5 (SIPSER, 2007).

4. Linguagens reconhecidas por Máquinas de Turing

As linguagens reconhecidas por alguma MT são as **linguagens recursivamente enumeráveis**, também chamadas de reconhecíveis por Turing. As decididas por alguma MT que sempre para são as **linguagens recursivas**, ou decidíveis. Na hierarquia de Chomsky, as classes estão encaixadas:

Regulares ⊂ Livres de contexto ⊂ Sensíveis ao contexto ⊂ Recursivas ⊂ Recursivamente enumeráveis

4.1 Exemplos

- Toda linguagem regular e toda linguagem livre de contexto é reconhecida por uma MT.
- $L = \{ a^n b^n c^n \mid n \geq 0 \}$: não é livre de contexto (Unidade 4), mas a MT da Seção 2.3 a decide usando a fita como memória, marcando o que já foi conferido.
- Executar algoritmos gerais sobre entradas codificadas, como somar números em binário: programas e dados são cadeias de símbolos na fita.

Observação: no exemplo $a^n b^n c^n$, a linguagem é **decidível**, e não apenas reconhecível, pois a máquina sempre para.

4.2 O papel da fita

Diferentemente da pilha, cujo acesso é só ao topo, na MT os símbolos marcados (X, Y, Z) permanecem na fita e a cabeça pode voltar para relê-los. É por isso que a MT compara três quantidades, ou quantas forem necessárias (HOPCROFT; MOTWANI; ULLMAN, 2002; LEWIS; PAPADIMITRIOU, 2004).

4.3 Variantes e Tese de Church-Turing

Várias variantes da MT foram estudadas: com várias fitas, com fita bidimensional, com cabeça que pode ficar parada e com **não determinismo**. Todas têm o mesmo poder computacional que o modelo básico, embora possam diferir em eficiência (SIPSER, 2007; DIVERIO; MENEZES, 2011).

Outros modelos independentes, como o cálculo lambda (CHURCH, 1936), as funções recursivas e as linguagens de programação usuais, também são equivalentes à MT. Essa convergência sustenta a **Tese de Church-Turing**: a noção intuitiva de algoritmo coincide com o que uma MT pode executar. Por envolver uma noção intuitiva, a tese não é um teorema demonstrável, mas é amplamente aceita.

Existe ainda a **Máquina de Turing Universal**, capaz de receber a descrição de uma MT M e uma palavra w e simular M sobre w. Essa ideia antecipa o computador de programa armazenado (TURING, 1936).

5. Limitação: linguagens que uma máquina de Turing não decide

Embora seja um modelo muito poderoso, a MT não decide toda linguagem. Uma linguagem é decidível quando existe uma MT que aceita toda palavra da linguagem, rejeita toda palavra que não pertence a ela e sempre termina. É indecidível quando não existe MT que decida corretamente todos os casos. Pela Tese de Church-Turing, isso significa que nenhum algoritmo resolve o problema.

5.1 Por que existem problemas indecidíveis

O conjunto de todas as MTs é enumerável, pois cada máquina se descreve por uma cadeia finita. Já o conjunto de todas as linguagens sobre um alfabeto é não enumerável. Há, portanto, mais linguagens do que máquinas, e a maioria das linguagens nem sequer é reconhecível.

5.2 O problema da parada

O problema da parada pergunta: dado um programa e uma entrada, o programa terminará sua execução? Como linguagem:

HALT = { ⟨M, w⟩ | M é uma MT que para com a entrada w }

Não existe algoritmo geral que responda corretamente a essa pergunta para todos os programas e entradas possíveis: HALT é indecidível (TURING, 1936). A ideia da demonstração, por contradição, é a seguinte:

Passo	Argumento
1	Suponha que exista uma MT H que decida HALT: H aceita ⟨M, w⟩ se M para com w, rejeita caso contrário, e sempre para.
2	Construa D, que recebe ⟨M⟩, executa H sobre ⟨M, ⟨M⟩⟩ e faz o oposto: entra em loop se H diz que M para, e para se H diz que M não para.
3	Execute D sobre ⟨D⟩. Se D para, H diz que D para e, por construção, D entra em loop. Se D não para, H diz que D não para e, por construção, D para.
4	Nos dois casos há contradição. Logo, H não existe.

Observação: o resultado não diz que é impossível saber se algum programa específico termina; muitos são analisáveis. Ele diz que não existe **um único algoritmo geral** que acerte para todos. Além disso, HALT é **reconhecível**: basta simular M sobre w e aceitar se a simulação parar. Ele não é decidível porque, quando M entra em loop, a simulação também não termina. HALT é, portanto, um exemplo de linguagem reconhecível e não decidível.

5.3 Consequências

A indecidibilidade se estende a outros problemas por **redução**: se um problema A permitisse resolver HALT, A também seria indecidível. Assim se prova, por exemplo, que não é possível decidir se dois programas são equivalentes. O **Teorema de Rice** generaliza esses resultados: toda propriedade não trivial da linguagem reconhecida por uma MT é indecidível (HOPCROFT; MOTWANI; ULLMAN, 2002). Na prática, ferramentas de análise de programas precisam abrir mão de exatidão total, por aproximações ou por restrição a classes de programas.

6 Resumo

Conceito	Ideia principal
Máquina de Turing	controle finito, fita ilimitada e cabeça de leitura e escrita; formaliza a noção de algoritmo
Transição	depende do estado e do símbolo lido; define o símbolo escrito, o movimento (L/R) e o novo estado
Comportamentos	aceitar, rejeitar (parar sem aceitar) ou entrar em loop (nunca parar)
Reconhecer	aceitar exatamente as palavras da linguagem (classe RE)
Decidir	aceitar, rejeitar e sempre parar (linguagens recursivas)
Poder	inclui as linguagens regulares e livres de contexto e resolve $a^n b^n c^n$
Limite	o problema da parada é indecidível, embora seja reconhecível

7. Erros frequentes

- Rejeitar ≠ loop → rejeitar para; loop nunca termina
- Nem toda não aceita é rejeitada → pode entrar em loop
- Decidível ≠ reconhecível → decidível $\subset$ reconhecível
- Indecidível ≠ não reconhecível → problema da parada é reconhecível
- Transição: lê → escreve → move (ordem importa)

- $\Sigma \neq \Gamma$ → entrada ≠ fita
- Branco (␣) não está em Σ
- Não existe Z_0 na MT (isso é de pilha)
- Variações da MT → mesmo poder (só muda eficiência)
- Church-Turing = tese (não teorema)
- Erro comum: não dá pra prever *todos* os programas, mas alguns sim

8. Aplicações em Computação

A Máquina de Turing define formalmente o que é um algoritmo e permite provar que certos problemas não têm solução algorítmica. O problema da parada e o Teorema de Rice explicam por que compiladores, antivírus e analisadores de código não conseguem verificar, de forma exata e geral, se um programa termina ou se dois programas são equivalentes; por isso usam aproximações. Linguagens como C, Java e Python têm o mesmo poder da MT e herdam suas limitações, e é por isso que servidores e juízes online impõem limites de tempo. A Máquina de Turing Universal antecipa interpretadores, máquinas virtuais e o computador de programa armazenado, e o número de passos de uma MT é a base das classes P e NP.

9. Questões de fixação

Questão 1. Quais são os três componentes de uma Máquina de Turing? Explique a função de cada um.

Questão 2. Escreva a definição formal $M = (Q, \Sigma, \Gamma, \delta, q_0, F)$ e explique a diferença entre Σ e Γ.

Questão 3. Explique a diferença entre rejeitar e entrar em loop. Dê um exemplo de máquina que entra em loop.

Questão 4. Explique a diferença entre reconhecer e decidir uma linguagem.

Questão 5. Usando a MT da Seção 2.3, simule a palavra aabbcc e indique se ela é aceita.

Questão 6. Usando a mesma MT, simule abcc e explique por que ela é rejeitada.

Questão 7. Construa uma MT que troque cada 0 por 1 e cada 1 por 0 em uma cadeia de {0,1}* e pare ao encontrar o branco.

Questão 8. Explique por que o problema da parada é reconhecível, mas não decidível.

Questão 9. Um colega afirma: "Como o problema da parada é indecidível, é impossível verificar se qualquer programa termina." Aponte o erro.

10. Síntese da unidade

A Máquina de Turing é composta por controle finito, fita infinita e cabeça de leitura/escrita, sendo definida formalmente por $M=(Q,\Sigma,\Gamma,\delta,q0,F)M = (Q, \Sigma, \Gamma, \delta, q_0, F)$. A cada passo, ela lê um símbolo, escreve outro, move a cabeça e muda de estado, podendo aceitar, rejeitar ou entrar em loop. Essa distinção é essencial para diferenciar reconhecer (pode não parar) de decidir (sempre para). A MT é mais poderosa que modelos anteriores e consegue resolver problemas como $a^n b^n c^n$, e todas as suas variações têm o mesmo poder computacional. A Tese de Church-Turing afirma que tudo que é computável pode ser executado por uma MT, mas ainda assim existem limites, como o problema da parada, que é indecidível.

REFERÊNCIAS

CHURCH, Alonzo. *An unsolvable problem of elementary number theory*. **American Journal of Mathematics**, v. 58, n. 2, p. 345–363, 1936.

DIVERIO, T. A.; MENEZES, P. B. *Teoria da Computação: máquinas universais e computabilidade*. Porto Alegre: Bookman/UFRGS.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introduction to Automata Theory, Languages, and Computation*. 3. ed. Pearson, 2006.

HOPCROFT, John E.; MOTWANI, Rajeev; ULLMAN, Jeffrey D. *Introdução à Teoria de Autômatos, Linguagens e Computação*. Rio de Janeiro: Elsevier.

LEHMAN, Eric; LEIGHTON, F. Thomson; MEYER, Albert R. *Mathematics for Computer Science*. Massachusetts Institute of Technology. Disponível em: MIT OCW – Mathematics for Computer Science (Readings). Acesso em: 29 set. 2026.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elements of the Theory of Computation*. 2. ed. Prentice Hall, 1998.

LEWIS, Harry R.; PAPADIMITRIOU, Christos H. *Elementos de Teoria da Computação*. Porto Alegre: Bookman.

MENEZES, P. B. *Linguagens Formais e Autômatos*. Porto Alegre: Bookman/UFRGS.

MIT OPENCOURSEWARE. *Mathematics for Computer Science — Chapter 7: Relations and Partial Orders*. Disponível em: MIT OCW – Chapter 7 Relations. Acesso em: 29 set. 2026.

MIT OPENCOURSEWARE. *Automata, Computability, and Complexity*. Massachusetts Institute of Technology. Disponível em: MIT OCW – Theory of Computation. Acesso em: 29 set. 2026.

ROSEN, Kenneth H. *Discrete Mathematics and Its Applications*. 8. ed. New York: McGraw-Hill Education, 2019.

SIPSER, Michael. *Introduction to the Theory of Computation*. 3. ed. Boston: Cengage Learning, 2012.

SIPSER, Michael. *Introdução à Teoria da Computação*. São Paulo: Cengage Learning.

TURING, Alan M. *On computable numbers, with an application to the Entscheidungsproblem*. **Proceedings of the London Mathematical Society**, série 2, v. 42, p. 230–265, 1936.